

北京邮电大学

Beijing University of Posts and Telecommunications

计算理论课程论文



从计算理论视角分析程序自动终止性检测问题

以现实软件系统为背景的形式化建模与不可判定性讨论

学院: 网络空间安全学院

姓名:

班级:

学号:

邮箱:

目录

摘要	4
Abstract	4
1 引言	5
1.1 选题原因：现实软件系统为何需要终止性检测	5
1.2 论文研究的主要目标与核心研究问题	6
1.3 文章结构与写作路线说明	6
2 计算理论基础：从状态机到图灵机的统一视角	7
2.1 计算模型的基本概念：状态、配置与状态转移	7
2.2 形式化语言与判定问题	9
2.3 图灵机模型回顾及其代表性	10
2.4 可判定性、可识别性与枚举器	10
2.5 复杂性理论视角：可判定并不等于可用	11
2.6 归约思想与不可判定性证明范式	12
3 现实软件系统中的终止性检测问题	13
3.1 IDE 与静态分析工具中的终止性提示	13
3.2 CI/CD 与代码审查中的终止性风险	14
3.3 操作系统与服务端程序中的资源耗尽问题	14
3.4 嵌入式与安全关键系统中的终止性需求	15
4 程序自动终止性检测问题的形式化建模	16
4.1 从程序到图灵机的编码方法	16
4.2 终止性判定问题 (PTP) 的形式化定义	17
4.3 停机问题 (HALT_{TM}) 的定义与解释	18
4.4 “终止”与“接受”的等价性	18
4.5 终止性问题的若干变体	20
4.5.1 实例终止性问题	20
4.5.2 总终止性问题	21
4.5.3 有界终止性问题	21
5 不可判定性分析：从停机问题到终止性检测	23
5.1 不可判定性证明的基本思路	23
5.2 从停机问题到终止性问题的归约构造	24

5.3	形式化证明：反证法与判定器假设	24
5.4	不可判定性的关键原因分析	26
5.5	Rice 定理视角下的终止性问题	28
5.6	个人思考：不可判定性为何具有工程意义	29
6	理论结论对工程实践的影响	30
6.1	不可判定性结论的正确理解	30
6.2	工程策略一：限制程序模型	31
6.3	工程策略二：保守近似与不完备分析	32
6.4	工程策略三：启发式与经验规则	34
6.5	工程策略四：人工注解与交互式证明	35
6.6	现实工具中的终止性分析实践	36
7	状态机视角下的终止性分析工具链	38
7.1	控制流图与状态空间探索	38
7.2	符号执行与终止性分析	40
7.3	抽象解释与不动点计算	41
7.4	运行时检测与监控机制	41
7.5	Trace、Profiler 与终止性定位	42
7.6	个人思考：状态空间爆炸与分析边界	43
8	一个可执行的简化模型：终止性检测的模拟实现	43
8.1	设计目标与建模假设	43
8.2	简化程序语言与执行语义	44
8.3	解释器的状态机实现	45
8.4	三类终止性检测策略的实现	46
8.4.1	完全模拟执行	47
8.4.2	有界步数执行	47
8.4.3	基于规则的静态检测	47
8.5	结果与案例分析	48
8.6	个人思考：工程工具的设计取舍	50
9	个人综合思考与扩展应用	51
9.1	计算理论对理解程序分析极限的价值	51
9.2	终止性检测中的正确性、完备性与可用性权衡	52
9.3	自己设计的应用场景与工程启示	53
9.3.1	代码审查与自动化质量控制	53

9.3.2	安全关键系统中的可验证编程子集	54
9.3.3	在线服务中的资源预算与有界执行	54
9.3.4	智能合约与脚本沙箱的步数限制	55
9.3.5	批处理任务与数据管道的终止性治理	55
10	结论	56
10.1	程序自动终止性检测问题的理论总结	56
10.2	不可判定性对现实软件系统设计的启示	57
10.3	局限与可能扩展	58
10.4	课程思想在本文中的体现与收获	58
	致谢	61
A	符号与缩写	62
B	Toy 语言补充：语法与语义要点	63
B.1	语法（BNF）与示例程序	63
B.2	小步操作语义	63

摘要

程序是否终止直接影响软件系统的可靠性与资源占用。本文从状态机视角形式化“程序在给定输入上是否终止”，将其表述为判定问题 PTP，并通过停机问题 HALT_{TM} 的多对一归约证明其不可判定性；进一步结合 Rice 定理说明终止性作为语义性质的普遍边界。在工程层面，本文对限制模型、保守近似与资源有界化等策略进行对照讨论，并借助一个可执行的 toy 解释器对三类检测策略的行为差异做直观比较，说明理论边界如何落到工具设计取舍上。

关键词： 计算理论；程序终止性；不可判定性；归约；程序分析

Abstract

Program termination directly affects software reliability and resource usage. From a state-machine perspective, we formalize instance termination as the decision problem PTP and prove its undecidability via a many-one reduction from the Halting Problem HALT_{TM} ; Rice's Theorem further clarifies termination as an undecidable semantic property. On the engineering side, we compare strategies such as model restriction, conservative approximation, and resource bounding, and use an executable toy interpreter to contrast three termination-checking strategies, showing how theoretical limits translate into practical design trade-offs.

Keywords: theory of computation; program termination; undecidability; reduction; program analysis

1 引言

1.1 选题原因：现实软件系统为何需要终止性检测

在现实软件系统中，“程序是否会在有限时间内结束”不是单纯的理论问题，而是直接牵涉可靠性、安全性与资源管理的工程需求。现代软件运行在多任务、多用户乃至分布式环境中，某段代码在特定输入下无法终止，会导致 CPU 长时间占用、内存持续增长、线程池耗尽或任务队列阻塞，进而引发性能退化、服务不可用甚至安全事件。比如在线服务中，攻击者可构造输入诱导程序进入非终止路径，形成拒绝服务风险；在嵌入式与控制系统里，任务未能按时结束会带来实时性失败，可能造成更严重的物理后果；在自动化测试与持续集成（CI）流程中，单个测试用例的非终止会拖垮流水线，使质量保障机制失效。

围绕“终止性”，工业界与学术界发展了许多工具与方法：静态分析器尝试在代码层面发现可能的死循环；模糊测试与运行时监控用超时机制捕获“疑似不终止”的行为；形式化验证在关键模块上追求可证明的终止；资源预算机制则在系统层面直接约束执行，例如步数限制、时间片、Gas 机制。它们的共同点是要么覆盖面有限、结论偏保守，宁可误报，要么需要人工注解与交互式证明。矛盾点在于工程上很想要“通用、自动且完全正确”的终止性检测器，但现实里总有程序绕开任何固定规则。智能合约的 Gas 机制，本质上就是“资源有界化”这一策略 [4]。

计算理论给出了理解这类矛盾的统一视角。围绕图灵机、判定问题与不可判定性的经典结论告诉我们：某些看起来“应该能自动判断”的目标，实际上会撞到通用计算的边界。终止性检测就是最典型的例子之一，它不仅有明确的现实背景，也自然牵连到形式化建模、语言刻画与归约证明等方法 [1, 2]。本文就从计算理论角度讨论程序自动终止性检测：一方面回答“能否完全自动判定”的理论问题，另一方面把结论落回到软件系统与工具链设计上，看看边界如何影响工程取舍。

进一步地，从“形式化描述”的角度看，终止性检测问题有一个很适合写成课程论文的特点：它既能被明确地定义为一个二值判定任务，又能自然地映射到不同层次的计算模型与工程约束之上。例如，在实际开发流程中，我们常见的“IDE 可能死循环提示”“CI 超时失败”“线上服务请求熔断/限流”“智能合约 Gas 耗尽回滚”等，都可以被视为对同一风险的不同处置方式。有的尝试在静态阶段给出保守结论，有的通过运行时资源上界把无界行为转化为可控失败。

基于上述动机，全文按“现象 → 模型 → 边界 → 工程策略”的顺序组织：先把现实中的“卡死/不退出”抽象为统一的判定问题，再用不可判定性结论给出理论边界，最后回到工程层面解释为何成熟系统往往采用多策略组合的工具链，并且需要明确分析的适用前提与失败模式。本文的目标不是把结论写成口号，而是把这条从理论到实践的推理链条讲清楚，便于在具体系统里做设计取舍。

在写作取材上，本文力求在形式化定义、不可判定性结论与工程实践之间建立对应关系：形式化定义用于刻画现实系统中的潜在风险，不可判定性结论则体现为工具设计中不可避免的能力边界。通过这样的安排，本文希望读者在理解相关问题理论上为何不可判定的同时，也能进一步认识在这些限制条件下，工程上仍然可行的策略与实践路径。

1.2 论文研究的主要目标与核心研究问题

本文以终止性检测这一现实需求为背景，尝试用计算理论的概念与方法把问题讲清楚：如何形式化建模、如何证明边界、以及边界对工程实践意味着什么。围绕这一目标，全文集中回答以下问题：

- (1) **形式化建模问题**：如何将现实中的“程序执行”抽象为计算理论中的对象，使“是否终止”能够被精确定义为一个判定问题？在这一过程中，程序、输入、运行步骤与停止条件分别对应计算模型中的哪些要素？
- (2) **可判定性问题**：是否存在一个对任意程序与任意输入都正确的通用算法，用以判定该程序是否会终止？若不存在，该“不可能性”结论如何通过课程中常用的不可判定性证明方法得到？
- (3) **工程解释问题**：终止性检测在一般意义下不可判定，是否意味着现实工具与实践没有价值？相反，现实系统中的静态分析、符号执行、运行时监控与资源限制等手段，如何在“可用性、成本与正确性”之间做出取舍，从而在不可判定性的阴影下仍然获得可用的分析能力？

围绕这些问题，本文希望做两件事：在理论层面把终止性检测的计算本质与能力边界说清楚；在工程层面解释现实工具为何不可避免地带有前提或不完备，并据此讨论系统该如何设定“自动化分析能保证到什么程度”。

1.3 文章结构与写作路线说明

全文采用“从理论走到实践、再回到理论”的写作路线：从计算理论的基础概念出发，引出现实软件系统中的终止性问题；再通过形式化建模与不可判定性分析，说明终止性检测在通用计算模型下的极限；随后讨论这些结论对工程实践与工具设计的启示，并用一个可执行的简化模型做直观对比；最后回到课程思想与方法论的总结与反思。

各章节之间按照“理论基础 → 现实问题 → 形式化分析 → 工程应对 → 扩展应用 → 总结反思”的逻辑顺序层层展开，其整体结构与写作路线如图 1 所示。

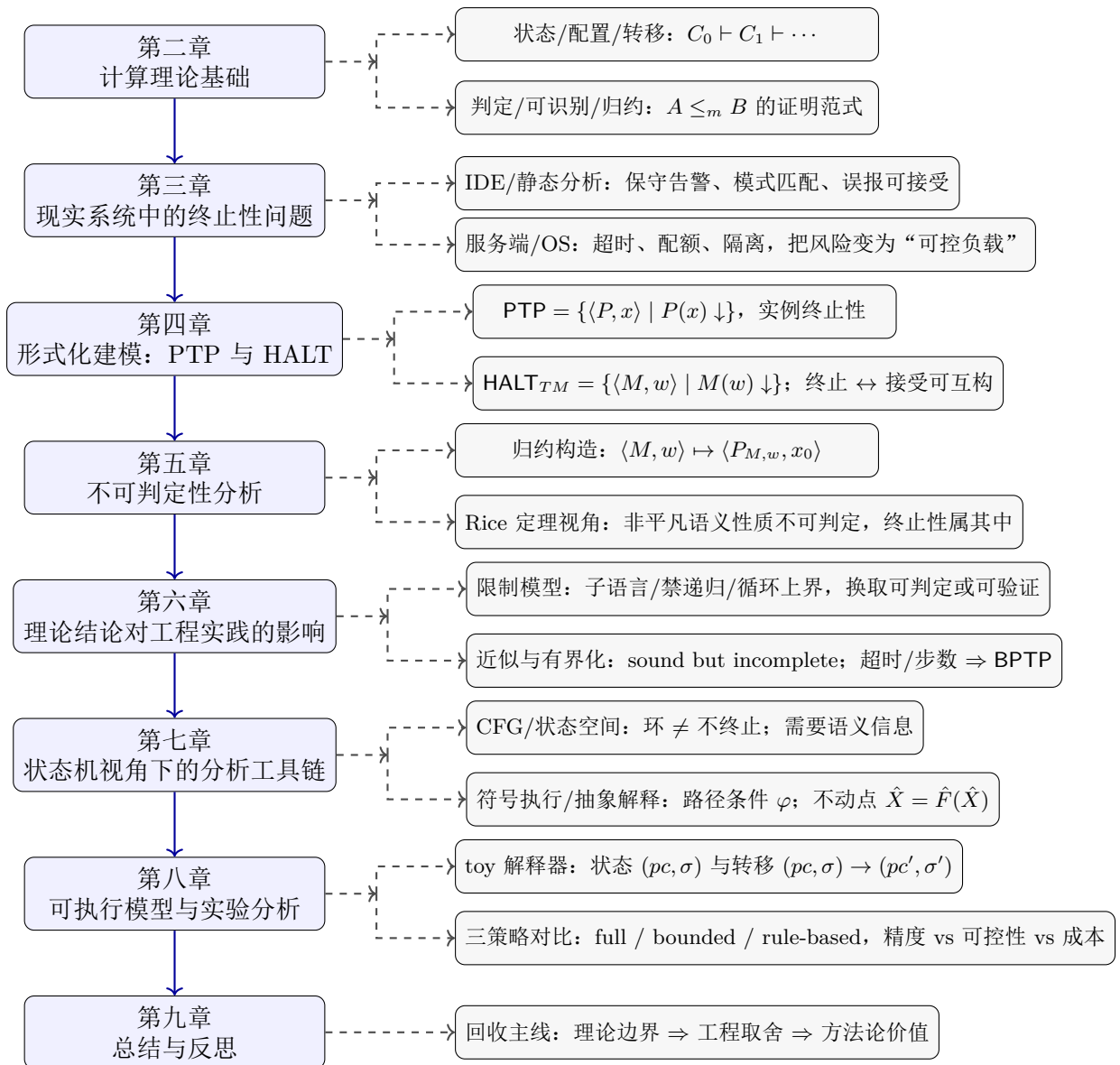


图 1: 本文整体结构与写作路线

2 计算理论基础：从状态机到图灵机的统一视角

2.1 计算模型的基本概念：状态、配置与状态转移

在计算理论中，对“计算过程”的研究并不直接依赖具体硬件实现或某种编程语言细节，而是通过抽象的计算模型刻画计算行为的共性结构。无论是有限自动机、下推自动机，还是图灵机与现实程序的抽象语义，状态 state、配置 configuration 与状态转移 transition 都是描述计算过程的三个基本概念。它们提供了一种统一语言，使我们能够将“程序在运行”这一动态过程转化为可被形式化分析的数学对象。

1. 状态 (state): 对系统内部模式的抽象。直观地说，状态用于刻画系统在某一时刻所处的内部“模式”。例如，在程序执行过程中，当前指令位置 program counter、局

部变量取值、堆对象引用关系、调用栈内容、寄存器值等都可以被视为状态的一部分。为了便于推理，计算模型通常假设存在一个状态集合 Q ，它可能有限，也可能可枚举，系统在任一时刻处于某个 $q \in Q$ 。在自动机模型中，状态常常只表示控制层面的信息，例如当前位于哪个控制状态；而在更强的模型中，如图灵机或程序语义模型，状态往往需要结合“外部存储”的内容才能完整刻画瞬时情形。

2. 配置 (configuration): 对一次瞬时描述的“全信息”刻画。为了更加精确地描述计算过程，计算理论引入配置 configuration 作为系统某一时刻的完整瞬时描述。与“状态”相比，“配置”通常包含更多信息：不仅包括控制状态，还包括外部存储的内容及读写头位置等，例如磁带、栈或内存。

以单带图灵机为例，一个常见的配置表示可以写成 $C = (q, u, a, v)$ ，其中 $q \in Q$ 为当前控制状态； uav 表示当前磁带内容，省略无限空白部分； a 为读写头当前指向的符号； u 与 v 分别表示读写头左右两侧的符号串。此时配置 C 刻画了图灵机在该时刻的全部信息：它不仅知道“机器处于哪个状态”，还知道“磁带上写了什么、头在什么位置”。

类似地，对现实程序进行抽象时，配置也可以写成更一般的形式，例如 $C = (\text{pc}, \sigma)$ ，其中 pc 表示当前程序位置，可理解为控制点； σ 表示存储状态，即把变量或内存位置映射到具体值的环境。这种记法在后文讨论控制流图、符号执行与抽象解释时会反复出现。

3. 状态转移：用一步计算刻画“动态”。 计算过程本身由配置之间的状态转移关系刻画。若一个配置 C_2 可以通过一次合法计算步骤由配置 C_1 得到，则记为 $C_1 \vdash C_2$ ，其中 $\vdash$ 可以理解为“一步推导/一步执行”。

为了描述多步执行，通常引入 $\vdash$ 的传递闭包：记 $C_1 \vdash^* C_2$ 表示 C_2 可由 C_1 经有限步到达，这里的步数可以为 0；记 $C_1 \vdash^+ C_2$ 表示至少一步到达。这样，“可达性 (reachability)” 就被自然形式化为：给定初始配置 C_0 ，所有可达配置集合为 $\text{Reach}(C_0) = \{C \mid C_0 \vdash^* C\}$ 。

从结构上看，这等价于把计算模型表示成一个有向图或转移系统 $\mathcal{T} = (\mathcal{C}, \rightarrow)$ ，其中 $\mathcal{C}$ 是配置集合， $\rightarrow$ 是一步转移关系。终止性问题因此被转化为“从初始配置出发的路径是否会在有限步内到达停机配置”的图论式判定。

为了让这一记号更直观，可以用一个具体的图灵机转移作为例子。若转移函数满足 $\delta(q_0, a) = (q_1, b, R)$ ，则对应配置转移为 $(q_0, u, a, v) \vdash (q_1, ub, v_1, v_2)$ ，也就是在读写头处把符号 a 改写为 b ，控制状态从 q_0 变为 q_1 ，并向右移动一步。类似地，在程序语义中，一条赋值语句也可以被刻画为配置从 $\langle \text{pc}, \sigma \rangle$ 到 $\langle \text{pc}', \sigma' \rangle$ 的一步转移。

4. 执行序列与终止：终止性问题的自然入口。 由此，一个程序（或某个计算模型）的执行过程可以被视为一个配置序列 $C_0 \vdash C_1 \vdash C_2 \vdash \dots$ 。若存在某个 t 使得 C_t 是停止配置，例如进入停机状态或到达接受/拒绝配置，则该执行序列在有限步后终止；反

之，若对任意 t 都存在后继配置 C_{t+1} ，则形成无限执行路径。终止性检测问题的核心，就是判断给定初始配置 C_0 的执行是否会在有限步内到达停止配置。

这种以状态/配置与转移为核心的描述方式，为后续关于控制流图的状态空间探索、符号执行的路径扩展、抽象解释的不动点计算，以及终止性不可判定性的归约证明，提供了统一且自然的形式化基础。

在计算理论中，对“计算过程”的研究并不直接关注具体的硬件实现或编程语言细节，而是通过抽象的计算模型刻画程序执行的本质结构。其中，状态、配置与状态转移是描述计算过程的三个基本概念。

直观地说，状态用于刻画系统在某一时刻所处的内部情形。例如，在程序执行过程中，当前指令位置、变量取值以及调用栈内容等，都可以被视为系统状态的一部分。为了便于形式化分析，计算理论通常将这些信息统一抽象为一个有限或可枚举的状态集合。在更精确的描述中，计算模型往往使用配置（configuration）来表示计算的完整瞬时描述。以图灵机为例，一个配置可以表示为 (q, u, a, v) ，其中 q 表示当前控制状态， uav 表示磁带内容， a 是当前读写头所指向的符号， u 与 v 分别表示读写头左右两侧的符号串。配置刻画了图灵机在某一时刻的全部信息。

计算过程本身则被描述为配置之间的状态转移。若一个配置 C_2 可以通过一次合法计算步骤由配置 C_1 得到，则记为 $C_1 \vdash C_2$ 。由此，程序或计算模型的执行过程可以被视为一个配置序列 $C_0 \vdash C_1 \vdash C_2 \vdash \dots$ 。该序列可能在某一接受或停止配置处终止，也可能无限延续。这种以状态转移为核心的描述方式，为后续关于“是否终止”的讨论提供了自然的形式化基础。

2.2 形式化语言与判定问题

为了系统地研究“是否存在算法解决某类问题”，计算理论通常将各种问题统一刻画为形式化语言。给定一个有限字母表 Σ ，其上的语言 L 被定义为所有有限字符串集合 Σ^* 的一个子集，即 $L \subseteq \Sigma^*$ 。在这一抽象下，语言中的每一个字符串都可以被理解为某个问题实例的编码表示。

在该框架中，一个判定问题对应于这样一个任务：给定任意输入字符串 $w \in \Sigma^*$ ，判断 w 是否属于语言 L 。换言之，判定问题关心的是一个二值结论，即“是”或“否”，而不涉及额外的输出信息。这种二值化的设定使得问题的可解性能够被清晰地形式化描述。

与之对应，若存在一台图灵机 M ，对任意输入 w 都能够在有限时间内停机，并且当 $w \in L$ 时接受、当 $w \notin L$ 时拒绝，则称语言 L 是可判定的（decidable）。此时，图灵机 M 被称为语言 L 的判定器。可判定性的核心要求不仅在于“给出正确答案”，还在于“对所有输入都保证停机”。

这种将问题转化为语言成员关系判定的方式，使得各种看似不同的问题能够在统一

的数学框架下进行比较和分析。现实中的问题往往并非天然以字符串形式出现，因此一个关键步骤在于编码：将程序、输入数据或系统描述这种问题实例有效地表示为有限字符串。只要这种编码过程本身是可计算的，便不会影响判定问题的理论性质。

在这一意义下，“某个程序是否会终止”可以被形式化为：给定一个描述程序及其输入的字符串，判断该字符串是否属于某个特定语言。这一抽象步骤将现实世界中的程序行为问题转化为标准的判定问题，使其能够被纳入计算理论的统一分析框架之中，并为后续关于可识别性、不可判定性以及归约关系的讨论奠定基础。

2.3 图灵机模型回顾及其代表性

图灵机是计算理论中最经典、也是最具代表性的计算模型之一。一台图灵机可以形式化定义为七元组 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$ ，其中 Q 是有限状态集合， Σ 是输入字母表， Γ 是磁带字母表（满足 $\Sigma \subseteq \Gamma$ ）， δ 是状态转移函数， q_0 为初始状态， q_{acc} 与 q_{rej} 分别表示接受与拒绝状态。

状态转移函数 δ 通常定义为 $\delta : (Q \setminus \{q_{acc}, q_{rej}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ ，即根据当前状态与读写头所读符号，决定下一状态、写入符号以及读写头的移动方向。接受状态与拒绝状态不再具有外出的转移，这一点在形式化上保证了一旦进入停机状态，计算便不可继续。

尽管图灵机模型在结构上极为简洁，但其计算能力却足以刻画现实中所有“可有效计算”的过程。根据丘奇-图灵论题（Church-Turing Thesis），任何在直觉意义下可由算法完成的计算，都可以由某台图灵机模拟完成。正因如此，在讨论程序终止性等问题时，将现实程序抽象为图灵机并不会削弱问题的普遍性，也不会人为引入额外限制。

从状态机视角来看，图灵机可以被理解作为一种具有无限存储能力的状态转移系统：有限的控制状态与无限磁带共同构成其配置空间，而执行过程正是配置之间的状态转移序列。这一理解为后文将“程序执行”与“状态空间探索”“终止性路径”联系起来提供了重要桥梁，使得终止性问题能够在统一的状态转移框架下进行分析。

2.4 可判定性、可识别性与枚举器

在形式语言框架下，并非所有语言都具有相同的可计算性质。除了可判定语言之外，计算理论还引入了可识别语言（recognizable language）的概念。若存在一台图灵机 M ，使得当 $w \in L$ 时 M 在有限时间内接受 w ，而当 $w \notin L$ 时 M 可以拒绝或永远不停止，则称语言 L 是图灵可识别的，亦称为递归可枚举语言。

可判定语言与可识别语言之间存在严格的包含关系： $\text{Decidable} \subsetneq \text{Recognizable}$ 。直观上，可判定语言要求算法对所有输入都必须停机并给出正确答案；而可识别语言仅保证“正实例最终会被确认”，对负实例则不要求给出有限时间内的结论。这种不对称

性在终止性问题中尤为关键，因为“无法判定”往往正是由于算法在某些输入上无法保证停机。

这一区分可以用停机问题给出非常直观的例子。语言 $\text{HALT}_{TM} = \{\langle M, w \rangle \mid M(w) \downarrow\}$ 是可识别的：给定 $\langle M, w \rangle$ 后，只需直接模拟 M 在输入 w 上的运行；若 M 的确在某一步停机，则模拟会在有限时间内观察到停机并接受。可识别性在这里对应一种“可证实性”：一旦停机发生，证据（停机的那一步）是可被观察到的。然而，对于不属于 HALT_{TM} 的输入（即永不停止的机器），模拟过程无法在有限时间内给出“永不停止”的证据，因此可能永远运行下去。

一个重要的等价刻画是：语言 L 可判定，当且仅当 L 与其补语言 $\bar{L}$ 都是可识别的。换言之，若一个语言的正例与反例都可以被半判定，那么就可以通过并行模拟的方式构造出一个对所有输入都停机的判定器。由此可见，许多不可判定问题的本质并非“完全无法处理”，而是其否定情形无法被有效识别。

这里的“并行模拟”可以理解为一种 dovetailing：同时运行识别器 R_L 与 $R_{\bar{L}}$ ，轮流给它们分配逐步增长的模拟预算。因为任意输入要么属于 L 要么属于 $\bar{L}$ ，总会有一个识别器最终接受，从而整体过程对所有输入都能停机。这个构造也提供了一个理解“不可判定”的视角：当我们只能识别正例而无法识别反例时，就不可能把识别器升级为总能停机的判定器。

要点：可识别

可识别语言强调“正例可被确认”：当性质成立时，算法最终能找到证据并停机；当性质不成立时，算法可能永远找不到证据而无法停机。终止性问题的困难正来自这一点：终止是一个可观察事件（到达停机配置），但“不终止”缺乏在有限时间内必然出现的反证。

2.5 复杂性理论视角：可判定并不等于可用

复杂性理论提醒我们：即使某个问题是可判定的，也不意味着它在工程上“可用”。复杂性理论按时间/空间等资源对可判定问题进行进一步分层，例如多项式时间类 P 与非确定性多项式时间类 NP ，以及大量通过归约刻画的 NP -完全问题 [1]。这与本文后续将使用的“归约”方法在形式上高度一致：在可计算性理论中，归约用于传递不可判定性；在复杂性理论中，归约用于传递“难解性”，如 NP -完全性。

这一视角对终止性分析同样重要：例如本文讨论的有界终止性 $BPTP$ 是可判定的，并且在“给定步数上界 k ”的前提下可以通过至多 k 步模拟直接求解；但当状态空间增大、路径分支爆炸或需要引入更精细的语义信息时，即使问题仍在可判定范围内，也可能因指数级开销而失去工程可行性。这正是现实工具链普遍采用“分层策略 + 有界化兜底”的重要原因之一：它们不仅在与不可判定性边界对齐，也在与复杂性带来的资源

约束对齐。

更一般地，许多“看似只是多跑一会儿就能判定”的分析任务，其实会迅速演化为高复杂度的问题：例如可达性、模型检查、约束求解等核心子问题在很多模型里都处于 PSPACE、EXPTIME 甚至更高的复杂性层级。对终止性工具而言，这意味着即使把目标改写为可判定版本（如有限状态抽象或有界探索），仍然可能因为最坏情况开销过大而不可用，因此必须依赖剪枝、抽象合并、启发式排序与增量求解等工程技巧。

要点：两类边界共同塑造工具形态

可计算性理论给出“能不能做”的边界（如不可判定性）；复杂性理论给出“做得起吗”的边界（资源爆炸）。现实终止性分析工具往往同时被两类边界塑形：一方面必须接受不完备或有界化，另一方面还要在可判定范围内继续做近似与分层，以满足性能与可用性。

与可识别语言密切相关的是枚举器（enumerator）的概念。一个枚举器是能够在无限运行过程中，按照某种顺序输出语言中所有字符串（可能伴随重复）的图灵机。经典结果表明：一个语言是图灵可识别的，当且仅当存在一个枚举器能够枚举该语言。这一等价关系从“生成”的角度刻画了可识别性：虽然我们未必能对任意输入立即判断其是否属于语言，但可以保证语言中的每一个元素最终都会被枚举出来。

这一视角对于理解终止性问题尤为有用。许多与程序行为相关的性质（如“存在一条会终止的执行路径”）往往是可识别但不可判定的：一旦找到满足条件的证据即可确认，但若该证据不存在，算法可能永远搜索下去而无法停机。这种“半可判定性”正是后文不可判定性分析的重要基础。

2.6 归约思想与不可判定性证明范式

为了证明某个问题不存在判定算法，计算理论中通常采用归约（reduction）方法。其基本思想是：通过建立问题之间的可计算映射关系，将一个已知困难的问题转化为目标问题的实例，从而比较二者在计算难度上的相对关系。直观地说，若能够利用问题 B 的判定算法来解决问题 A ，而 A 已被证明不可判定，则必然导致矛盾。

形式上，若存在一个可计算函数 f ，使得对任意输入 x 都有 $x \in A \iff f(x) \in B$ ，则称问题（或语言） A 可多对一归约（many-one reduction）到 B ，记为 $A \leq_m B$ 。该定义意味着：任何能够判定 B 的算法，都可以通过先计算 $f(x)$ 再调用该算法，从而判定 A 。因此，在这种情况下， B 在判定意义下至少与 A 一样困难。

归约方法在不可判定性证明中的作用，正是通过这种“困难性传递”来完成的。常见的证明思路可以概括为：选取一个已知不可判定的问题 A ，构造可计算变换 f 使得 $x \in A \iff f(x) \in B$ ；若假设 B 可判定，则对任意 x 先计算 $f(x)$ 再判定 B ，就能判定

A, 与已知结论矛盾。

在众多不可判定问题中，停机问题 HALT_{TM} 是最常用、也是最基础的归约起点之一。大量关于程序语义性质的不可判定性结论，都是通过将停机问题归约到目标问题而得到的。这是因为停机问题本身直接刻画了“程序是否在有限时间内结束”这一核心计算行为，天然适合作为分析程序终止性及相关性质的基准问题。本文后续关于程序自动终止性检测问题的理论分析，正是遵循这一经典归约证明范式展开的。

3 现实软件系统中的终止性检测问题

3.1 IDE 与静态分析工具中的终止性提示

在软件开发的早期阶段，终止性风险往往以“静态提示”的形式出现在集成开发环境（IDE）和静态分析工具中。这类工具不实际执行程序，而是根据源代码结构给出预警。例如，缺乏显式退出条件的循环结构，或递归调用中缺少明显递减参数（ranking function）的函数，常会被标记为“可能无法终止”。

从分析方法上看，IDE 中的终止性提示通常基于对程序控制结构的近似建模。工具会为程序构建控制流图（Control Flow Graph, CFG），并分析其中是否存在不受约束的循环路径，即从某一控制点出发，在不满足任何终止条件的情况下可以反复回到自身的执行路径。若分析器在 CFG 中发现此类路径，便倾向于发出潜在非终止的警告。

这里要分清：这类提示并不等价于对终止性的严格判定。循环或递归结构是否真的会跑不完，往往取决于运行时输入、变量变化规律以及外部调用语义，而这些因素在静态阶段通常只能被近似建模。因此，IDE 的提示更像是保守预警：宁可误报“可能不终止”，也很难做到对所有终止程序都给出肯定结论。

为了让“静态提示”更有解释性，许多工具会尝试寻找终止性证明中常见的结构性证据，例如规模下降（size decrease）或变元函数（ranking function）。直观地说，若能为循环或递归找到一个映射 $R(\cdot)$ ，使得每次迭代/递归都满足 R 严格下降且有下界（例如自然数上的严格下降），则可以据此构造一个终止性证明。工程工具往往只能在受限的模式下自动识别这类证据：例如仅对线性递减变量、单调更新、简单的循环条件等情形给出较强结论；而对涉及复杂分支、外部调用或非线性更新的代码则倾向于保守处理。

要点：终止性“提示”与“证明”的区别

IDE/静态分析中的告警通常回答的是“这里存在可能导致不终止的结构或路径”，而不是“该程序必然不终止”。当工具给出“可能不终止”时，更多是在提醒开发者补充终止条件、缩小输入范围或增加保护措施；当工具给出“看起来会终止”时，也应理解为在其模型假设与规则覆盖范围内的结论，而非对通用程序的绝对保证。

3.2 CI/CD 与代码审查中的终止性风险

在持续集成与持续交付（CI/CD）流程中，终止性问题以更加工程化的形式出现。自动化构建、测试与部署任务通常按照预先定义的流水线顺序执行，其中任何一个步骤若无法在合理时间内结束，都可能阻塞后续阶段，降低开发与交付效率。

与 IDE 中的静态提示不同，CI/CD 系统通常并不尝试在理论层面判断某个程序是否必然终止，而是通过设置超时阈值、资源配额等机制，对潜在的非终止行为进行约束。例如，单元测试或集成测试任务若在限定时间内未完成，系统便会强制中断其执行并标记为失败。这种做法并非解决终止性判定问题本身，而是通过对执行时间和资源的有界化限制，使“可能不终止”的行为在工程上变得可控。

在代码审查阶段，开发者也可能基于经验对存在非终止风险的代码提出修改建议，例如要求补充明确的循环不变式或终止条件。然而，这种人工判断依赖于审查者对程序语义的理解，难以形成统一且自动化的判定标准。CI/CD 与代码审查中的这些实践表明，在复杂软件系统中，终止性问题更多是通过流程设计与工程约束来缓解，而非通过严格的自动判定来解决。

从工程治理角度看，CI/CD 往往会把“终止性风险”拆解为一组可操作的机制组合：其一，为测试与任务引入分层超时（单测/集成/端到端分别设定阈值），并将超时视为可追踪的缺陷而非偶发事件；其二，在超时发生时尽量保留定位信息（如线程栈、热点函数、日志片段、输入样本），避免“只知道卡死但无法复现”；其三，把高风险任务隔离到独立资源池或沙箱环境中运行，防止单个任务耗尽共享执行器；其四，对外部依赖（网络、数据库、第三方 API）设置调用超时与断路器，避免“外部调用不返回”被误判为内部死循环。上述做法本质上是在系统层面构造一个“可控失败”的终止性兜底：即使无法静态判定是否终止，也能保证流水线整体不会无限等待。

3.3 操作系统与服务端程序中的资源耗尽问题

在操作系统和服务端程序中，终止性问题往往以资源耗尽的形式显现出来。一个未能终止的进程或线程，可能持续占用处理器时间、内存空间、文件描述符或网络连接，从而影响系统中其他任务的正常执行，甚至引发系统级故障。

操作系统通常不尝试回答“某个进程是否会终止”这一判定问题，而是用调度与资源管理机制限制非终止行为带来的影响。例如，时间片轮转调度可以防止单个进程长期占用 CPU；对内存、句柄和网络资源设置上限，可以抑制资源泄漏的扩散。这些机制不依赖对程序终止性的分析，而是通过隔离与限额，把潜在的非终止行为转化为可控的系统负载。

在现代服务端实践中，这类“有界化兜底”通常会以更具体的工程形态出现：容器或 cgroup 可以对 CPU/内存做硬限制；应用层通过请求超时、并发上限与背压机制防止请求堆积；进程层通过 watchdog/健康检查与自动重启，把异常卡死从“永久占用”转化为“短暂扰动”；任务系统通过幂等重试与死信队列避免重试风暴。在这些机制下，系统设计者并不需要回答“某段代码是否必然终止”，而是把关注点转移到“最坏情况下是否可恢复、是否能维持整体服务质量”。

这与计算理论给出的边界是对齐的：当“是否终止”无法被统一判定时，系统就把问题改写为“最多消耗多少资源”，并用有界化管理保证整体可用性与稳定性。

3.4 嵌入式与安全关键系统中的终止性需求

与通用计算环境不同，航空航天、工业控制和医疗设备这种嵌入式系统与安全关键系统对终止性往往提出更为严格的要求。在此类系统中，程序不仅需要最终终止，还必须在规定的時間界限内完成执行，否则可能导致实时性失败，甚至引发严重的安全事故。

为了满足这一需求，工程实践中通常采取对程序模型进行严格限制的策略。例如，限制或禁止递归结构，要求循环具有显式且可验证的执行上界，或采用经过认证的编程规范和安全子语言。在实时系统中，这类分析往往与最坏情况执行时间（Worst-Case Execution Time, WCET）分析相结合，用以保证程序在时间维度上的可预测性。

这些限制在一定程度上降低了程序的表达能力，但换来了终止性和时间行为的可分析性。这一现象表明，在对终止性要求极高的场景中，工程实践往往主动放弃一般程序的灵活性，以换取在受限模型下对终止性与时间行为的可判定性。这种取舍与计算理论中“通过限制计算模型获得可判定性”的思想高度一致。

从实现路径看，安全关键领域常把“终止性保证”分摊到多个层面：在语言与规范层面，通过限制动态分配、限制指针别名、约束循环结构来减少不可预测性；在架构层面，通过静态调度与周期任务模型（例如固定周期、固定优先级）降低交互复杂度；在验证层面，通过代码审查规则、模型检查与形式化证明把关键循环/递归的终止证据落到可审计的工件上。这样做的核心思想仍然是：不在图灵完备的“任意程序”上追求全自动终止性判定，而是把系统设计成终止性可被证明或被上界约束的形态。

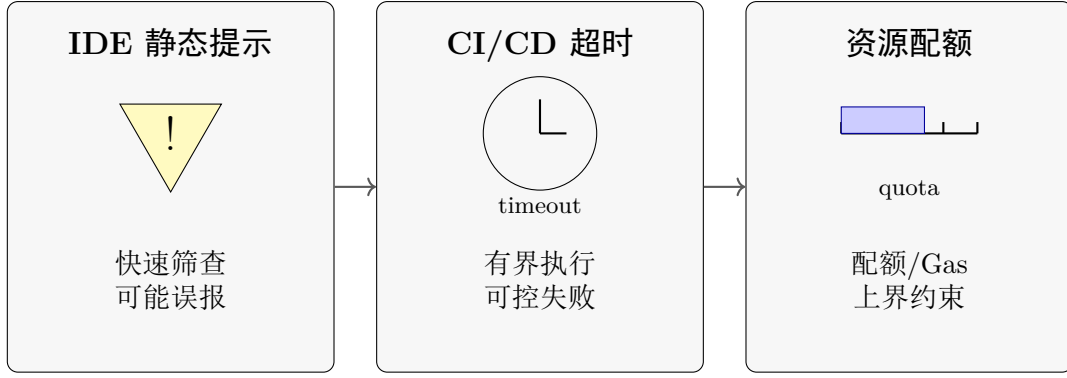


图 2: 现实系统中终止性风险的常见触发场景示意

4 程序自动终止性检测问题的形式化建模

上一章从工程实践角度展示了终止性问题在不同软件系统中的具体表现。本章把这些现象进一步抽象到计算理论的框架中，给出形式化建模：引入统一的符号体系与计算模型，把终止性讨论转化为标准判定问题，为后续不可判定性分析提供明确对象。

4.1 从程序到图灵机的编码方法

为了在计算理论中讨论程序的终止性，需要先说明“程序”这一现实概念如何映射为形式化的计算对象。根据丘奇-图灵论题（Church-Turing Thesis），任何在直觉意义下可由算法完成的计算过程，都可以由某台图灵机模拟。因此，在理论分析中把程序抽象为图灵机并不会削弱问题的一般性，反而提供了统一建模的标准计算模型。

设 P 表示一个使用某种高级编程语言编写的程序， x 表示其输入数据。我们假设存在一种可计算的编码方式，用以将程序 P 表示为一台图灵机 M_P ，并将输入 x 表示为图灵机的输入串 w_x ，使得如下等价关系成立：

$$P \text{ 在输入 } x \text{ 上终止} \iff M_P \text{ 在输入 } w_x \text{ 上停机.}$$

从形式化角度看，该过程可以理解为存在一个可计算的编码映射 $\langle \cdot \rangle$ ，使得 $\langle P \rangle \mapsto M_P$ ， $\langle x \rangle = w_x$ ，并且程序的执行语义能够被图灵机的配置转移关系忠实模拟。

为了保证这一建模的合理性，编码与模拟过程一般需要满足三点：编码过程本身可计算；程序的每一步语义执行可由图灵机有限步转移模拟；程序终止应对应图灵机进入某个停机配置。

从状态机的角度看，程序执行过程可以被理解为一个状态转移系统，而图灵机正是一种具有无限存储能力的状态机模型。二者在“状态与转移”层面上的对应关系如图 3 所示。

在后续讨论中，本文将默认采用这种抽象方式，将程序及其执行行为统一视为图灵

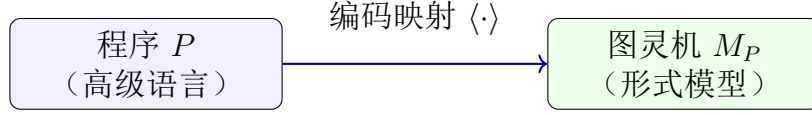


图 3: 现实程序执行语义与图灵机计算模型之间的抽象对应关系

机及其计算过程，从而在统一的形式化框架下分析程序自动终止性检测问题。

4.2 终止性判定问题（PTP）的形式化定义

在完成从现实程序到图灵机的抽象之后，可以对“程序是否终止”这一问题给出严格的形式化定义。计算理论中通常通过形式化语言来刻画判定问题，因此程序自动终止性检测也可以被表述为某个语言的成员关系判定问题。这种表述方式使得终止性问题能够在统一的理论框架下，与其他经典判定问题进行比较与归约。

定义 4.1 (终止性判定问题). 定义终止性判定问题 (*Program Termination Problem, PTP*) 对应的语言为

$$PTP = \{ \langle P, x \rangle \mid P \text{ 在输入 } x \text{ 上终止} \},$$

其中 $\langle P, x \rangle$ 表示程序 P 及其输入 x 的一种有效编码。

在该定义中，符号 $\langle P, x \rangle$ 表示将程序描述与输入数据共同编码为一个有限字符串，这是将现实中的程序行为问题引入形式语言框架的关键步骤。PTP 中的每一个元素都对应一个具体的程序实例，而对应的判定任务则是：给定任意输入字符串 $\langle P, x \rangle$ ，判断其是否属于语言 PTP。

从计算模型的角度看，该定义将程序终止性问题明确转化为关于执行过程是否在有限步内结束的问题。若将确定性程序执行抽象为状态转移系统，则 $\langle P, x \rangle \in PTP$ 当且仅当从初始配置 $C_0(P, x)$ 出发，存在某个终止配置 C_h 使得

$$C_0(P, x) \vdash^* C_h.$$

若采用非确定性语义，则需要进一步区分“存在一条终止路径”与“所有路径都终止”两种不同的终止性定义。本文在后续讨论中默认程序语义为确定性单路径语义，以避免歧义。

需要强调的是，PTP 描述的是实例终止性问题，即程序在某一个固定输入上的终止行为，而非程序在所有可能输入上的终止性。这一区分在理论与工程实践中都具有重要意义：实例终止性关注单个执行是否结束，而全局终止性则是一个更强、也更严格的性质，常见于函数式编程或安全关键系统的分析中。

正是由于 PTP 以“程序 + 输入”的形式刻画单次执行行为，它与经典的停机问题

在结构上高度相似。后续章节将进一步证明，在通用计算模型下，PTP 无法存在一个对所有输入都保证停机且给出正确答案的判定算法，从而揭示程序自动终止性检测在理论层面所面临的根本限制。

4.3 停机问题 (HALT_{TM}) 的定义与解释

在计算理论中，最经典、也是最基础的终止性相关问题是停机问题 (Halting Problem)。该问题在图灵机模型提出后即被证明不可判定：不存在一个对所有输入都保证停机且给出正确答案的判定算法。停机问题由此成为不可判定性理论的起点 (教材表述可见 [1, 2])。

定义 4.2 (停机问题). 定义停机问题对应的语言为

$$\text{HALT}_{TM} = \{\langle M, w \rangle \mid \text{图灵机 } M \text{ 在输入 } w \text{ 上最终停机}\}.$$

其中, $\langle M, w \rangle$ 表示图灵机 M 的描述与输入串 w 的一种有效编码。判定 HALT_{TM} 的任务是：给定任意图灵机及其输入，判断该图灵机在执行过程中是否会在有限步内进入停机状态。

需要注意的是，这里的“停机”并不区分图灵机是接受还是拒绝输入。只要计算过程在有限步内结束，无论最终进入接受状态 q_{acc} 还是拒绝状态 q_{rej} ，都被视为停机。相反，若图灵机在输入 w 上产生无限执行序列，则称其在该输入上不停止。

从执行语义的角度看， $\langle M, w \rangle \in \text{HALT}_{TM}$ 当且仅当从初始配置 $C_0(M, w)$ 出发，存在某个停机配置 C_h ，使得

$$C_0(M, w) \vdash^* C_h.$$

若不存在这样的配置，则对应一条无限延续的配置转移路径。

与前文定义的终止性判定问题 PTP 相比， HALT_{TM} 在结构上高度相似。前者关注“程序在给定输入上的终止行为”，后者关注“图灵机在给定输入上的停机行为”。由于现实程序已经通过可计算的方式被抽象为图灵机，这两类问题在计算理论框架下具有一致的本质结构。正因如此， HALT_{TM} 常被用作分析程序终止性问题的基准问题，其不可判定性结论也可以通过归约传递到更一般的程序终止性检测问题中。

在后续分析中，本文将利用 HALT_{TM} 的不可判定性，构造从停机问题到程序终止性判定问题的归约，从而严格证明 PTP 在通用计算模型下同样不可判定。

4.4 “终止”与“接受”的等价性

在形式化讨论中，“终止（停机）”与“接受”是两个相互独立的概念：前者只要求计算过程在有限步内停止（进入 q_{acc} 或 q_{rej} 皆可），后者则要求机器最终进入特定的接

受状态 q_{acc} 。然而，在不可判定性分析与归约证明中，这两类概念在计算难度上可以被视为等价的：我们可以通过可计算的构造在二者之间相互转化，从而使证明能够在“停机/接受/拒绝”等表述之间自由切换，而不改变结论的本质。

从“停机”到“接受”的转化构造。给定任意图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$ ，我们构造一台新的图灵机 M' ，使得对任意输入 w 有

$$M \text{ 在输入 } w \text{ 上停机} \iff M' \text{ 在输入 } w \text{ 上接受}.$$

构造思想非常直接：让 M' 在输入 w 上模拟 $M(w)$ 的运行；一旦检测到 M 进入任一停机状态（无论是 q_{acc} 还是 q_{rej} ）， M' 都不再保留原结果，而是统一转入自己的接受状态 q'_{acc} 并停机接受。

更形式化地说， M' 的状态集合可以包含 M 的所有状态以及一个新的接受状态 q'_{acc} 。在转移函数层面， M' 与 M 在非停机状态上的转移完全一致；当模拟到 M 的停机状态时， M' 额外定义一条“汇合”转移，使其进入 q'_{acc} ：

$$\delta'(q, a) = \begin{cases} \delta(q, a), & q \notin \{q_{acc}, q_{rej}\}, \\ (q'_{acc}, a, R), & q \in \{q_{acc}, q_{rej}\}. \end{cases}$$

（这里的 R 仅表示任意方向移动一次，实际并不重要；关键在于进入 q'_{acc} 后停机。）

正确性说明。若 M 在输入 w 上经过有限步停机，则模拟过程会在某一时刻到达 q_{acc} 或 q_{rej} ，此时 M' 立即转入 q'_{acc} 并接受，因此 M' 接受 w 。反之，若 M 在 w 上不停止，则 M' 的模拟永远不会遇到停机状态，也就不可能进入 q'_{acc} ，因此 M' 不会接受（事实上会无限运行）。由此得到上面的等价关系成立。

为何这一等价性对不可判定性证明重要。这一转化说明：在归约证明中，我们常常无需严格区分“语言成员（接受）”与“是否停机（终止）”。例如，停机问题 $HALT_{TM}$ 关注“是否停机”，而许多标准语言判定问题更自然表述为“是否接受”。借助上述构造，我们可以将“停机”类性质转化为“接受”类性质，或者反向转化，从而统一证明对象并复用不可判定性结论。

结论

在多对一归约与不可判定性分析中，“停机/终止”与“接受”在计算难度上可等价处理；只要构造是可计算的，证明可以在两种表述之间切换而不影响结论。

图4给出了这一等价构造的直观关系：通过对机器进行可计算改造，将“是否停机”的判定转化为“是否接受”的判定。

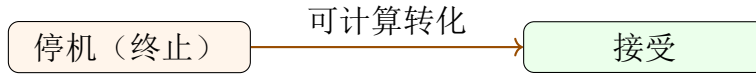


图 4: 通过可计算构造将“停机”转化为“接受”

4.5 终止性问题的若干变体

在实际研究与工程实践中，“终止性”并非单一概念，而是可以根据关注对象与约束条件的不同，形成多种形式化问题。不同变体之间在表达能力与判定难度上存在显著差异，其可判定性结论也随之发生变化。对这些变体进行区分，有助于更清晰地理解终止性分析在理论与实践中的边界。

问题变体	形式化对象	理论性质（概述）	工程对应
实例终止性 PTP	给定 (P, x) 的一次执行	通用模型下不可判定，但“终止的正例”可通过模拟观察到	单个用例/请求是否卡死
总终止性 TOT	给定 P 的全输入行为	比 PTP 更强的全称性质，通用模型下不可判定	语言子集/规范保证“总终止”
有界终止性 BPTP	给定 (P, x, k) 的有限步行为	引入界限后可判定（有限步模拟）	超时、步数限制、Gas 机制

表 1: 终止性问题常见变体对比（概述）

4.5.1 实例终止性问题

实例终止性问题关注的是程序在某一固定输入上的终止行为，即本文前文所定义的程序终止性判定问题 PTP：

$$\text{PTP} = \{ \langle P, x \rangle \mid P \text{ 在输入 } x \text{ 上终止} \}.$$

从语义上看，实例终止性问题对应于单次程序执行路径的分析，其本质在于判断从初始状态出发的执行序列是否在有限步内到达某个终止状态。由于该问题不涉及对所有输入的全称量化，其形式上与停机问题 HALT_{TM} 最为接近。

实例终止性问题在工程实践中具有直接对应：例如，某个具体测试用例是否会卡死、某次服务请求是否会进入无限循环等。然而，正是由于其表达能力足以刻画任意图灵机在给定输入上的行为，该问题在通用计算模型下被证明是不可判定的。这也是本文后续不可判定性分析的主要研究对象。

需要强调的是，“不可判定”并不意味着实例终止性完全不可处理：对于会终止的实例，可以通过逐步模拟在观察到终止配置后给出肯定结论；困难在于不终止的实例缺乏可在有限时间内必然出现的“反例证据”。这正是现实中超时与监控机制普遍存在的原因：工程系统往往愿意在有限预算内做“尽力判断”，但必须接受“预算用尽仍无法断言是否不终止”的结果。

4.5.2 总终止性问题

与实例终止性问题相比，总终止性问题要求程序在所有可能输入上均能终止，其对应语言可以定义为

$$\text{TOT} = \{\langle P \rangle \mid \forall x, P \text{ 在输入 } x \text{ 上终止}\}.$$

该问题描述的是程序的一种全局语义性质，即程序是否对整个输入空间都具有终止性保证。由于该定义中包含对无限输入集合的全称量化，其判定难度显著高于实例终止性问题。

从计算理论角度看，TOT 不仅是不可判定的，而且其不可判定性可以通过对停机问题或 PTP 的归约得到。直观而言，若能够判定一个程序是否对所有输入终止，则可以通过构造特殊程序，将该能力用于解决单个输入上的停机问题，从而产生矛盾。

在工程实践中，总终止性问题通常只在受限模型或特定编程范式下被讨论。例如，某些函数式语言或受限子语言通过语法和类型系统约束，试图保证程序的总终止性，但这种保证往往以牺牲表达能力为代价。

从证明形式上看，总终止性往往需要构造“对任意输入都适用”的终止证据，例如结构递归、大小度量单调下降、或对循环次数给出显式上界等。这类证据一旦建立，通常不仅能说明“最终终止”，还常常附带更强的可预测性（如上界、复杂度估计）。因此，总终止性在工程上更像是一种通过设计获得的性质：先限制表达能力，再换取可验证的全局保证。

4.5.3 有界终止性问题

若在终止性判定中额外引入执行步数上界 k ，则可以定义有界终止性问题为

$$\text{BPTP} = \{\langle P, x, k \rangle \mid P \text{ 在输入 } x \text{ 上于 } k \text{ 步内终止}\}.$$

与前述两类问题不同，由于搜索空间被显式限制为至多 k 步的执行序列，该问题在理论上是可判定的：只需对程序执行进行有限步模拟，即可在有限时间内得出结论。这一性质并非源于程序本身的简化，而是来自对执行过程施加的有界性约束。

同时也要注意有界结论的解释方式：当回答 YES 时，这是一个强结论（确实在 k

步内终止)；当回答 NO 时，它只说明“在 k 步内未终止”，并不等价于“永不终止”。因此，有界化策略在工程上常被用作强保证 + 弱否定的组合：用 YES 提供可依赖的成功信号，用 NO 触发降级、隔离或人工介入，而不是把 NO 当作语义层面的否定证明。

引理 4.3 (有界终止性可判定). $BPTP$ 是可判定的。

证明. 给定任意输入 $\langle P, x, k \rangle$ ，我们在通用计算模型中对 $P(x)$ 的执行进行至多 k 步的逐步模拟：若在第 k 步之前到达终止配置则输出 YES，否则输出 NO。由于模拟步数被显式上界 k 限制，整个过程必然在有限时间内结束，且结论与 $BPTP$ 的定义一致，因此 $BPTP$ 可判定。□

推论 4.4 (有限状态系统的实例终止性可判定). 若程序（或系统）的配置空间是有限集合，则其实例终止性判定是可判定的。

证明. 在有限配置空间中，从初始配置出发的执行要么在有限步内到达终止配置，要么在有限步内出现重复配置并进入循环。通过对执行进行模拟并记录已出现的配置，最多在“配置数 + 1”步内即可做出判断，因此该终止性判定可判定。□

例 4.5 (有限状态带来可判定性). 若一个程序只包含有限个布尔变量（或枚举变量）且不进行动态内存分配，则其状态可被表示为“控制点 + 变量取值”的有限集合。此时判断“某个输入下是否会终止”就可以通过对有限状态空间的遍历完成：一旦到达终止状态则返回 YES；若遍历过程中出现重复状态则可判定其进入循环并返回 NO。

有界终止性问题的工程实践中具有广泛应用。例如，测试框架中的超时机制、操作系统中的时间片限制、区块链系统中的 Gas 上限，均可以被视为对终止性问题引入外部执行界限的实例。虽然这些机制并不能从根本上解决程序是否必然终止的问题，但它们通过限制执行资源，使潜在的非终止行为在工程上变得可控。

要点：把不可判定问题“改写”为可判定

不可判定性结论并不意味着工程上无能为力。常见做法是引入外部约束（如步数/时间/资源上界），把“无界终止性”替换为“有界终止性”或“有限状态可达性”，从而获得可判定且可落地的分析目标。

总体而言，这些终止性问题变体表明：终止性分析的可判定性高度依赖于所采用的计算模型与约束条件。通过限制输入范围、执行步数或程序表达能力，可以在一定程度上恢复可判定性，但代价是分析结果的适用范围受到限制。这一观察为后续从不可判定性结论出发讨论工程策略奠定了基础。

5 不可判定性分析：从停机问题到终止性检测

本章是全文的理论核心。我们将采用计算理论中最经典、最稳健的证明套路：从一个已知不可判定的问题出发，通过归约证明目标问题同样不可判定。具体而言，本文将以停机问题 HALT_{TM} 为基准，通过构造性归约证明终止性判定问题 PTP 不可判定，并进一步用 Rice 定理给出更一般的视角解释。

5.1 不可判定性证明的基本思路

在计算理论中，证明某个判定问题不可判定，并不依赖于对“所有可能算法”的穷举或否定，而是采用归约（reduction）这一间接但极为有力的方法。归约的核心思想在于：通过逻辑上严格的转换，将一个已知不可判定的问题嵌入到目标问题之中，从而证明目标问题至少同样困难。

直观而言，归约证明遵循如下逻辑模式：若存在一种自动算法可以判定问题 B ，而我们又能够将任意问题 A 的实例有效地转换为 B 的实例，使得二者的答案完全一致，那么就可以利用该算法间接判定 A 。因此，一旦 A 已知不可判定，就可以推出 B 同样不可判定。

形式化地，设 $A, B \subseteq \Sigma^*$ 为两个判定问题对应的语言。若存在一个可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$ ，使得对任意输入 x 都有 $x \in A \iff f(x) \in B$ ，则称 A 多对一归约到 B ，记为 $A \leq_m B$ 。这一关系刻画了判定难度之间的偏序关系：若 $A \leq_m B$ 且 B 可判定，则 A 也必然可判定。其逆命题同样成立：若 A 不可判定且 $A \leq_m B$ ，则 B 不可判定。

需要特别指出的是，归约函数 f 本身必须是可计算的。这一要求保证了归约过程不会在转换步骤中“偷偷引入”更强的计算能力。换言之，归约只能重新组织已有计算，而不能凭空解决原本困难的问题。正因如此，归约被视为不可判定性证明中最严格、也最具说服力的方法。

在本文的讨论中，归约的起点是经典的停机问题 $A = \text{HALT}_{TM}$ ，而目标问题则是程序自动终止性判定问题 $B = \text{PTP}$ 。我们的目标是构造一个可计算的变换 f ，将任意停机问题实例 $\langle M, w \rangle$ 转换为某个程序终止性实例 $\langle P_{M,w}, x_0 \rangle$ ，使得二者在逻辑上严格等价：

$$\langle M, w \rangle \in \text{HALT}_{TM} \iff \langle P_{M,w}, x_0 \rangle \in \text{PTP}.$$

需要注意的是，这里的程序 $P_{M,w}$ 并非一个“通用程序”，而是根据给定的图灵机 M 与输入 w 专门构造的程序实例。其核心思想并不在于实现复杂的工程功能，而在于通过程序执行过程精确模拟 M 在输入 w 上的计算行为。只要该程序在输入 x_0 上的终止性与 $M(w)$ 的停机性保持同步，归约即告成功。

图5给出了这一归约思路的整体结构示意。该图强调的并非具体实现细节，而是归

约中信息流与逻辑依赖关系：停机问题的实例经由可计算变换 f 被嵌入为程序终止性实例，其 YES/NO 判定结果在整个转换过程中保持不变。后续小节将基于这一总体框架，给出具体的归约构造，并完成对程序自动终止性检测问题不可判定性的形式化证明。

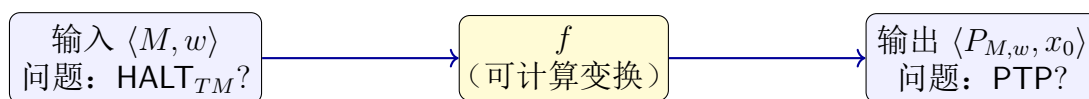


图 5: 从停机问题到程序终止性问题的归约证明总体结构

5.2 从停机问题到终止性问题的归约构造

下面给出归约的核心构造。给定任意停机问题实例 $\langle M, w \rangle$ ，我们构造一个新程序 $P_{M,w}$ ，使得 $P_{M,w}$ 的行为与 M 在输入 w 上的停机行为“绑定”。

构造目标。我们希望满足： M 在输入 w 上停机 $\iff P_{M,w}$ 在输入 x_0 上终止，其中 x_0 可以取一个固定的哑输入（例如空串 ϵ ），也可以取任意常量输入。

定义程序 $P_{M,w}$ 如下：

```

1 Program P_{M,w}(input x0):
2     simulate M on input w step by step
3     if the simulation of M halts then
4         halt (terminate)
5     else
6         continue simulation forever

```

该构造的关键点是： $P_{M,w}$ 并不真正依赖输入 x_0 的内容，它的“终止与否”完全由内部模拟的 $M(w)$ 是否停机决定。因此， $P_{M,w}$ 在 x_0 上终止，当且仅当 M 在 w 上停机。

为了更直观地理解这一绑定关系，图6给出了 $P_{M,w}$ 的控制流程结构。该图不是简单的“圈圈箭头”，而是明确刻画了“模拟器 + 停机分支 + 无限循环”的结构。

有效性说明。上述构造是可计算的：给定 $\langle M, w \rangle$ ，我们能够机械地写出一个模拟器程序，其内部硬编码 M 的转移函数并在磁带上初始化输入 w 。这与课程中“图灵机可以模拟任意图灵机”的思想一致。因此，映射 $f(\langle M, w \rangle) = \langle P_{M,w}, x_0 \rangle$ 是一个可计算函数。

5.3 形式化证明：反证法与判定器假设

在完成从停机问题到程序终止性问题的归约构造之后，可以采用计算理论中最为标准的反证法，对程序自动终止性判定问题的不可判定性给出严格证明。该证明的核心思想是：假设存在一个“理想的”终止性判定器，并展示这一假设将直接导致对停机问题的判定，从而产生矛盾。

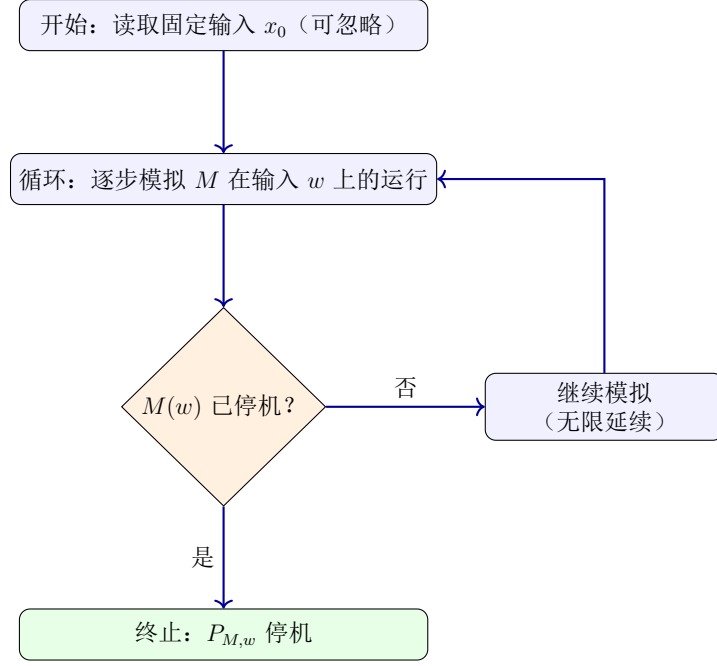


图 6: 归约构造中 $P_{M,w}$ 的控制流程结构

定理 5.1. 程序终止性判定问题 PTP 是不可判定的。

证明. 假设相反, PTP 是可判定的。根据可判定性的定义, 存在一台图灵机 D , 作为 PTP 的判定器, 使得对任意输入 $\langle P, x \rangle$, D 都在有限步内停机, 并且能够给出正确判断:

$$D(\langle P, x \rangle) = \begin{cases} \text{YES}, & \text{若程序 } P \text{ 在输入 } x \text{ 上终止,} \\ \text{NO}, & \text{若程序 } P \text{ 在输入 } x \text{ 上不终止.} \end{cases}$$

需要强调的是, 作为判定器, D 必须同时满足两个条件: 一是正确性, 即判断结果与实际终止行为一致; 二是全定义性, 即对所有输入都保证在有限时间内停机。这正是后续推导矛盾的关键所在。

现在考虑任意一个停机问题实例 $\langle M, w \rangle$, 其中 M 是一台图灵机, w 是其输入串。根据前一小节中给出的归约构造, 我们可以在有限时间内、通过可计算方式构造出一个程序 $P_{M,w}$, 并选取一个固定输入 x_0 , 使得

$$P_{M,w} \text{ 在输入 } x_0 \text{ 上终止} \iff M \text{ 在输入 } w \text{ 上停机.}$$

将构造得到的 $\langle P_{M,w}, x_0 \rangle$ 作为输入交给判定器 D , 并根据 D 的输出进行如下分析:

- 若 D 输出 **YES**, 则根据判定器的正确性, 有 $\langle P_{M,w}, x_0 \rangle \in PTP$, 即程序 $P_{M,w}$ 在输入 x_0 上终止。由归约构造的性质可知, 这当且仅当图灵机 M 在输入 w 上停机, 因此 $\langle M, w \rangle \in \text{HALT}_{TM}$ 。

- 若 D 输出 **NO**，则说明 $P_{M,w}$ 在输入 x_0 上不终止。同样由构造可知，这当且仅当 M 在输入 w 上不停机，因此 $\langle M, w \rangle \notin \text{HALT}_{TM}$ 。

由此得到一个对停机问题的判定过程：对任意输入 $\langle M, w \rangle$ ，先可计算地构造 $\langle P_{M,w}, x_0 \rangle$ ，再运行判定器 D 并据其输出给出结论。由于 D 对所有输入都保证停机，该过程整体上将成为 HALT_{TM} 的判定器。

然而，停机问题 HALT_{TM} 已被证明是不可判定的。这与上述推导得到“ HALT_{TM} 可判定”的结论直接矛盾。因此，最初关于“PTP 可判定”的假设不成立，从而可以得出结论：PTP 不可判定。□

结论

在通用计算模型下，不存在一个能够对所有程序与输入都给出正确判断、并且自身保证停机的通用程序终止性检测算法；程序自动终止性检测问题（PTP）在理论上是不可判定的。

5.4 不可判定性的关键原因分析

上述归约与反证证明给出了严格的不可判定性结论，但在论文写作中更值得回答的问题是：不可判定性究竟源于什么？换言之，为什么“判断程序是否终止”会成为一条计算理论的根本边界，而不是一个尚未被发明出来的高级算法尚可解决的工程难题？

本文认为，终止性不可判定性并非偶然，而是由程序计算的三类结构性特征共同导致的：通用表达能力使得问题至少与停机问题同样困难；终止性属于语义性质使其不能被有限的语法规则完全覆盖；而判定器必须对所有输入停机这一要求则放大了信息不对称，使得问题无法在有限时间内“排除所有可能”。下面分别展开说明。

（1）通用表达能力导致“嵌入停机问题”。程序之所以难以被统一判定，一个直接原因是现实程序通常具备通用计算能力：它们可以实现复杂控制流、递归、动态数据结构以及对外部输入的依赖。形式化地，只要程序模型能够模拟任意图灵机，就意味着：对任意图灵机 M 与输入 w ，都能构造一个程序实例，使其运行行为与 $M(w)$ 同步。于是，若存在对所有程序实例都正确的终止性判定器，就等价于对所有图灵机实例给出停机结论，从而不可避免地触及 HALT_{TM} 。

从“状态机/状态空间”视角看，通用程序的状态空间通常是可枚举但无穷的，例如由于堆内存、递归深度、整数变量取值范围等，终止性问题则对应于判断初始状态是否能在有限步内到达某个终止状态，即一种可达性问题：

$$\text{终止} \iff \exists \text{ 终止配置 } C_h, C_0 \vdash^* C_h.$$

当状态空间足够强大时，可达性判定会继承停机问题的不可判定性。这说明不可判定性

不是“算法能力不够”，而是问题本身已经包含了通用计算的全部困难性。

(2) 终止性是语义性质而非纯语法性质. 第二个关键原因在于：终止性并不是由源代码的局部语法形态决定的，而依赖程序在运行时的真实行为（即语义）。虽然一些明显的非终止模式可以被静态规则捕捉，例如 `while(true)`、缺乏递减参数的递归、或循环变量不更新等，但这些规则本质上仅覆盖有限的语法模式集合。

然而，在一般情形下，程序是否终止取决于输入数据与状态演化：同一段循环代码在不同输入下可能终止或不终止；终止条件可能隐藏在复杂的条件分支、外部函数返回值、甚至运行时环境交互之中。更重要的是，语义等价但语法形式不同的程序可以轻易绕过任何固定语法规则。换言之，任何试图“靠语法模式覆盖全部程序行为”的方法都会遭遇根本限制：因为终止性所描述的是程序计算行为的整体性质，而不是可以通过有限语法模板穷尽的局部特征。

这也解释了现实工具中的普遍现象：静态分析若想保持正确性（sound），往往必须变得保守并产生误报；若想减少误报，则不可避免地牺牲完备性（complete），甚至在某些情况下与不可判定性边界直接相撞。

(3) 判定器必须对所有输入停机：信息不对称的放大. 第三个原因与“判定问题”的定义本身有关。不可判定性讨论的对象不是任意程序分析器，而是判定器：它要求对所有输入都在有限时间内停机并给出 YES/NO 的确定答案。这一要求在终止性问题中尤其苛刻，因为终止性存在天然的信息不对称：

- 若程序确实终止，我们可以通过实际运行在有限时间内观察到终止，从而得到“YES”的证据；
- 若程序不终止，则不存在某个有限时间点能提供“永不终止”的决定性证据，因为任意有限前缀都可能只是更长计算的开始。

因此，终止性问题通常呈现“正例可证、反例难证”的结构：它更接近可识别而非可判定。这也解释了为何在实践中我们经常能做出半决定（semi-decision）：例如通过模拟执行或符号执行，当发现某条路径确实到达退出点时即可确认终止；但当迟迟找不到终止证据时，分析过程可能无限延续。

将上述三点合在一起，可以得到一个更完整的理解：终止性不可判定性并非来自某个“技巧性反例”，而是由通用程序模型的表达能力、语义性质的整体性以及判定器对停机保证的严格要求共同决定的。理解这一点对于后续工程讨论至关重要：工程实践并非“无视理论”，而是在不可判定性边界内，通过限制模型、有界化资源或采用保守近似等方式，构造出可用但不完美的终止性分析工具。

5.5 Rice 定理视角下的终止性问题

除了通过停机问题的直接归约得到不可判定性结论之外，程序终止性问题还可以置于 Rice 定理（Rice’s Theorem）这一更为一般的理论框架下理解 [1]。Rice 定理揭示了一类具有普遍性的不可判定性结论，其核心思想并不依赖于具体的构造技巧，而是源于对“程序语义性质”本身的抽象刻画。

Rice 定理的基本思想. Rice 定理指出：任何关于图灵机所计算对象（语言或部分函数）的非平凡语义性质，都是不可判定的。其中，“语义性质”强调性质只依赖于计算结果本身，而与机器的具体编码、状态数或转移书写方式等语法细节无关；“非平凡”则指该性质既不是对所有机器都成立，也不是对所有机器都不成立。换言之，只要一个性质真正区分了“不同计算行为”，且不退化为恒真或恒假，那么就不存在一个通用算法能够判定它。

为了避免概念混淆，需要指出：本文前文的实例终止性问题 PTP 以“程序 P + 输入 x ”为实例，其不可判定性最直接的证明方式仍然是通过 HALT_{TM} 的归约。Rice 定理在这里更适合用来解释为何一大类语义分析目标都天然困难，并为“总终止性”等全局语义性质给出一条更“模板化”的不可判定性推导路径。

注 5.2 (Rice 定理的使用边界). *Rice* 定理关注的是“关于程序所计算对象（语言或部分函数）的语义性质”，因此更适用于诸如总终止性（*totality*）、值域非空、输出恒为 0 等全局语义性质的不可判定性推导；对“给定输入是否停机”这类实例性质，更标准的路线仍是以 HALT_{TM} 为基准做归约证明。

总终止性作为 Rice 定理的典型例子. 考虑总终止性（*totality*）这一性质：

$$\mathcal{T}(P) : \text{“对所有输入 } x, P(x) \text{ 都会终止”}.$$

该性质只依赖于程序所计算的部分函数是否在所有输入上都有定义，因此属于语义性质；同时它是非平凡的（存在总是终止的程序，也存在在某些输入上不终止的程序）。由 Rice 定理可知，这类性质在通用计算模型下不可判定，这与前文关于 TOT 的讨论相一致。

Rice 定理对终止性问题的启示. 从 Rice 定理的视角看，终止性相关问题的不可判定性并非源于某个特殊构造或巧妙反例，而是通用计算模型中语义分析能力的结构性限制。只要程序模型具有足够的表达能力，能够描述任意复杂的可计算行为，那么任何试图对其运行语义给出统一、自动且完全正确判断的尝试，都会不可避免地遭遇不可判定性边界。

这一结论不仅涵盖本文所讨论的实例终止性问题 PTP，也同样适用于诸如“程序是否对所有输入终止”“程序是否存在某种特定运行行为”等更一般的语义判定问题。

Rice 定理所强调的是一种普遍现象：在通用计算模型下，关于程序行为本身的自动判定能力存在根本上不可逾越的限制。

从这一意义上说，终止性问题的不可判定性并非一个孤立结论，而是程序语义分析领域中更广泛不可判定性图景的具体体现。这一视角为后续从工程实践角度理解“为何只能近似分析终止性”提供了重要的理论背景。

5.6 个人思考：不可判定性为何具有工程意义

不可判定性结论在初看之下似乎是一种“悲观结果”，因为它否定了程序终止性进行完全自动判定的可能性。然而，从工程视角来看，这一结论并非消极限制，反而具有非常重要的指导意义：它为程序分析工具与系统机制所能达到的能力设定了清晰边界，使工程设计能够建立在现实可行的预期之上，而非理想化假设之中。

(1) 约束了工具的能力承诺。 不可判定性为分析工具的能力宣称划定了理论红线。任何声称“对任意程序和输入都能自动判断是否终止且永远正确”的工具，都与计算理论中的不可判定性结论直接矛盾。在工程实践中，成熟工具往往会明确其适用前提，例如仅对特定语言子集、特定控制结构或特定资源模型给出保证。不可判定性使得这种“能力边界声明”不再显得保守，而是成为一种必要且负责任的工程态度。

(2) 解释了为何必须引入限制或近似。 现实中的静态分析器之所以大量采用限制模型或保守近似，并非因为分析技术不足，而是因为终止性本身属于不可判定的语义性质。通过限制程序表达能力（如禁止递归、要求循环变量单调下降、引入结构化控制流），可以将分析问题限制在可判定的子空间；通过保守近似，宁可误报，也不漏报，可以在保持正确性的前提下提供有限但有用的分析结论。这些策略从理论角度看，正是对不可判定性边界的理性回应。

(3) 推动系统层面的替代方案：有界资源与隔离。 当终止性无法被完全判定时，系统设计往往转而采用机制性约束，将“是否终止”的问题替换为“最多消耗多少资源”的问题。超时限制、指令步数上界、内存配额、沙箱隔离等机制，本质上都是通过对执行过程施加有界性约束，使潜在的非终止行为在工程上变得可控。例如，对脚本执行设定时间或步数上限，可以将不可判定的终止性问题转化为可判定的有界终止性问题（参见第四章中 BPTP 的讨论）。这一思路体现了系统工程中“不解决问题本身，而是管理其影响”的典型设计哲学。

(4) 对软件工程教育与方法论的启示。 不可判定性还对软件工程教育与实践方法论提出了重要启示：在复杂系统中，追求“完全自动、完全正确”的程序语义分析往往是不

现实的。更具可行性的路线，是将形式化方法与工程约束相结合：在关键模块中采用可验证的编程子集或受限语言，在高风险代码中引入人工审查与交互式证明，在系统层面配合运行时监控与资源限制。这种“理论边界 + 工程折中”的思维方式，有助于培养对程序分析工具能力的理性预期。

总的来看，不可判定性并不是对工程实践的否定，而是一种基础性的边界意识。它帮助我们识别哪些目标在理论上做不到，从而把工程努力集中在边界之内、但确实能提升可靠性与安全性的设计与分析策略上。

6 理论结论对工程实践的影响

上一章证明了终止性判定问题 PTP 在一般意义下不可判定。这并不等于“工程上毫无办法”，而是说明：想要一个对任意程序与任意输入都完全正确、并且自身保证停机的终止性判定器，这个目标本身就不可能实现。现实系统之所以还能在一定程度上管理终止性风险，是因为工程实践接受了限制与折中：限制分析对象的表达能力；放弃完备性或放弃绝对可靠性；引入人为证明要素；或把问题改写为有界版本。本章围绕这些策略展开讨论，并结合常见工具实践解释其背后的理论逻辑。

6.1 不可判定性结论的正确理解

不可判定性结论在程序分析领域中经常被误读为“任何形式的自动分析都不可能”或“终止性检测在工程上没有价值”。事实上，这种理解过于极端，也并不符合计算理论所给出的真实含义。上一章所证明的不可判定性结论，针对的是一种非常强的目标：存在一个对所有输入都保证停机，并且对所有程序实例都给出完全正确判断的通用算法。换言之，不可判定性否定的是“通用性、完备性与全自动性”三者同时成立的可能性，而并非否定一切形式的终止性分析。

从工程实践角度看，只要放弃其中至少一个条件，终止性分析就仍然具有广泛而现实的应用空间。常见的工程出路主要包括以下三类。

(1) 限制分析对象的表达能力。 第一种思路是通过限制程序的表达能力，使终止性问题落入可判定或更易分析的范围。例如，将程序限定在某个受控的子语言中，禁止或严格约束递归、动态内存分配和非结构化控制流；或者要求循环结构满足可验证的单调下降条件。在这类受限模型下，程序的状态空间和执行路径结构更加规整，终止性往往可以通过静态方法得到严格证明。这一策略在安全关键系统、嵌入式系统以及部分函数式语言设计中被广泛采用。

(2) 接受不完备但可靠的分析结论。 第二种思路是放弃“对所有终止程序都能给出结论”的要求，仅保证分析结果的可靠性（soundness）：即当分析工具报告“程序一定终止”时，该结论在语义上是正确的；但对于某些实际上会终止的程序，工具可能无法给出肯定结论。这类分析通常基于保守近似，其本质是在不确定时选择“拒绝下结论”。在实际开发中，这种策略能够有效避免误判带来的安全风险，因而被大量静态分析器与形式化验证工具采用。

(3) 接受启发式与不可靠但实用的方法。 第三种思路则进一步放宽正确性要求，采用启发式规则或经验模型，对程序终止性进行快速但不完全可靠的判断。这类方法可能产生误报或漏报，但在规模巨大、变化频繁的工程环境中，往往能在可接受的成本下覆盖大量常见情形。例如，IDE 中的终止性警告、基于模式匹配的代码审查规则，均属于这一范畴。其价值并不在于“理论完美”，而在于对开发流程的辅助作用。

不可判定性还提示我们必须区分两类问题：无界判定问题与有界判定问题。前者要求对所有可能执行路径给出结论，因而往往不可判定；后者则在输入规模或执行资源上施加明确界限，从而恢复可判定性。例如，有界终止性问题

$$\text{BPTP} = \{ \langle P, x, k \rangle \mid P \text{ 在输入 } x \text{ 上于 } k \text{ 步内终止} \}$$

是可判定的，因为我们可以通过有限步模拟穷尽所有可能执行。这一理论观察恰好解释了为何“超时机制”“步数上限”“资源预算”等设计在现实系统中如此普遍：它们并非权宜之计，而是在不可判定性约束下的理性工程选择。

关键理解

因此，工程实践往往把目标调整为“可验证但不完备”或“有界但可控”：在受限模型、保守近似或资源上界的前提下给出稳定可用的结论，并明确哪些情况只能返回未知或触发兜底机制。

6.2 工程策略一：限制程序模型

一种最直接、也是在安全关键系统中最常见的工程策略，是通过限制被分析程序的表达能力，使其运行行为落入某些更可控、更易分析的计算模型中。该策略的核心思想并非试图解决一般意义下的终止性判定问题，而是在系统层面主动改变问题本身：只允许那些终止性可以被静态证明或形式化验证的程序进入运行环境。

在具体实践中，这类限制通常体现在程序结构、控制流形式以及动态特性的约束上，例如：

(1) 结构化循环与可验证上界：要求循环次数能够由某个整型变量单调递减（或递增）

并且存在明确的上下界，从而可以构造简单的排名函数（ranking function）证明循环必然终止；

- (2) **禁止或限制递归**：通过显式栈与迭代结构替代递归调用，或在语法与运行时层面限制递归深度，以避免隐式的无限调用链；
- (3) **限制动态特性**：禁止或严格约束反射、动态代码生成、任意指针算术等特性，使程序的控制流与状态空间在静态分析阶段更加可预测；
- (4) **受限脚本或 DSL**：在安全敏感或资源受限环境中使用领域特定语言（DSL），并在语言设计阶段内置终止性与资源使用约束，例如强制执行时间或步数预算。

从计算理论角度看，这些约束实质上是在削弱程序模型的计算能力，使其不再具有完整的图灵完备性。一旦程序模型被限制到某些较弱的计算模型，例如仅允许原始递归、结构化循环或有限状态控制。终止性问题往往重新变得可判定，或至少可以通过简单的语义推理得到保证。这一做法体现了工程实践中“用模型约束换取可验证性”的典型思路。

在安全关键系统中，这种策略尤为常见。此类系统通常宁可牺牲部分表达能力与开发灵活性，也要换取程序行为的可预测性与可验证性，从而降低潜在的系统性风险。

下面给出一个典型的、可以被静态证明终止的循环形态示例。该示例中，循环变量严格单调递减，并且具有显式的下界，因此可以构造简单的终止性证明。

```
1 int countdown(int n){  
2     // 前置条件:  $n \geq 0$   
3     while(n > 0){  
4         n--; // 单调递减, 且下界为 0  
5     }  
6     return 0; // 在有限步内必然终止  
7 }
```

在形式化验证中，上述循环可以通过将 n 作为排名函数来证明终止性：每次迭代都会使 n 严格减小，而 n 的取值域下界为 0，因此循环不可能无限执行。这类简单而可验证的结构，正是工程中通过限制程序模型来规避不可判定性影响的典型体现。

6.3 工程策略二：保守近似与不完备分析

当终止性问题在一般意义下不可判定时，工程实践中最常见的应对方式之一，是采用保守近似（conservative approximation）的分析策略。这类方法并不试图对所有程序实例给出结论，而是明确放弃完备性，转而保证分析结果的可靠性：只要工具给出“程

序一定终止”的结论，该结论在语义上就必须是正确的；但对于大量实际上会终止的程序，工具可能无法证明，从而只能返回“未知”或“可能不终止”。

这种分析策略通常被概括为“*sound but incomplete*”。这里 *sound* 表示可靠性：分析结果不会产生“假阳性”，也就是不会把一个实际不终止的程序误判为终止；*incomplete* 表示不完备：分析器可能无法识别所有真实的终止程序。这一取舍在安全和可靠性要求较高的场景中尤为重要，因为误判“终止”往往比误判“可能不终止”带来更严重的后果。

在终止性分析中，保守近似最常见的技术之一是尝试构造变元函数或排名函数 (ranking function)。形式上，分析器试图为循环或递归过程寻找一个映射

$$R : \text{State} \rightarrow W,$$

其中 W 是某个良序集合（如非负整数集），并且满足对任意连续执行状态

$$R(\text{state}_{t+1}) < R(\text{state}_t), \quad R \geq 0.$$

若这样的函数存在，则根据良序集合不存在无限下降序列的性质，可以形式化地推出该执行过程必然在有限步内终止。¹

需要注意的是，排名函数的存在性本身并非必然：某些终止程序的终止性依赖复杂的算术性质、非线性关系或跨多变量的协同变化，这些行为可能超出了分析器预设的函数模板或搜索空间。因此，当分析器无法找到合适的 R 时，并不能据此断言程序不终止，而只能得出“无法证明终止”的结论。

下面的示例展示了这一现象。该程序在实践中被广泛认为对所有正整数输入都会终止，但其终止性并不容易简单的静态规则捕捉：

```
1 int tricky(int n){
2     // n >= 1
3     while(n != 1){
4         if(n % 2 == 0) n = n / 2;
5         else n = 3*n + 1; // Collatz-like
6     }
7     return 0;
8 }
```

该循环的控制流并不存在显式的线性递减变量， n 在某些分支中甚至会显著增大。尽管大量实验表明该过程在实践中总会收敛，但其终止性涉及深层的数学问题，超出了大多数静态分析器的分析能力范围。因此，许多基于排名函数或简单规则的终止性分析

¹相关思想在终止性研究中有多种形式化变体，例如 Size-Change Principle[5] 等。

工具都会对该代码给出“未知”或“可能不终止”的结果。

这一例子清楚地展示了保守近似分析的典型后果：不完备性带来了误报或无法定论，但在分析器给出肯定结论时，终止性保证是可靠的。从工程角度看，这种取舍并非缺陷，而是一种有意识的设计选择。它允许分析工具在不可判定性边界内，仍然对大量常见程序结构提供有价值的自动化支持，同时避免因错误断言而引入潜在风险。

6.4 工程策略三：启发式与经验规则

除了保守近似分析之外，工程实践中还广泛存在一类更为务实的终止性检测方法：启发式与经验规则。这类方法并不试图在理论上保证正确性，而是接受一定程度的“不可靠”（unsoundness），以换取分析速度、实现成本和对常见错误模式的高覆盖率。

在具体实现上，这类启发式规则往往基于对程序结构的直观观察和长期工程经验总结，例如：

- 在循环体中检查关键控制变量是否发生变化，若变量在多次迭代中保持不变，则高度怀疑循环无法退出；
- 分析循环条件中涉及的变量，检查这些变量是否在循环体或相关调用中被更新；
- 对递归函数检查是否存在明显的“规模缩减参数”，例如参数值在递归调用中趋向某个基准情况；
- 对可能阻塞的 I/O 操作、锁等待或条件变量，引入超时机制或最大重试次数作为防护措施。

这类方法在 IDE、lint 工具和代码审查辅助系统中非常常见，其优势在于实现简单、运行开销低，并且能够在开发早期捕捉大量由疏忽或经验不足导致的非终止风险。对于初学者或快速迭代的项目而言，这种“高性价比”的反馈机制往往比复杂的形式化分析更具实用价值。

然而，与前两类工程策略相比，启发式方法在理论上并不提供任何完备性或可靠性保证。变量发生变化并不意味着循环一定会终止；递归参数缩减也可能在某些分支中被破坏；复杂控制流、外部依赖或数据相关行为都可能使启发式规则失效。因此，这类方法不可避免地会产生误判或漏检。

下面的示例展示了一段启发式规则能够有效捕捉的典型问题：循环条件依赖变量 x ，但循环体中从未修改该变量，因而在绝大多数语义下都无法退出循环。

```
1 int suspicious(int x){  
2     while(x > 0){  
3         // BUG: x never changes
```

```

4      // do something
5  }
6  return 0; // 实际上无法到达
7  }

```

对于这种代码，启发式分析工具即使在不进行复杂语义推理的情况下，也可以给出“高度疑似非终止”的警告，从而在早期阶段帮助开发者发现问题。

从工程视角看，启发式终止性检测并不是对不可判定性结论的挑战，而是对其现实影响的务实回应。这类方法不追求成为通用判定器，而是作为开发工具链中的“第一道防线”，与保守近似分析、模型限制和运行时机制等策略协同工作，共同降低非终止行为在实际系统中的风险。

6.5 工程策略四：人工注解与交互式证明

当系统对终止性具有极高的可靠性需求（例如操作系统内核、航空航天控制软件或加密协议实现中的关键模块）时，工程实践往往采用一条更为严格但成本更高的路线：引入人工注解与交互式证明。在这种策略下，程序的终止性不再完全依赖工具自动推断，而是由开发者显式提供证明所需的关键语义信息，例如循环不变式、变元函数或递归度量，再由验证工具对这些注解进行机械化检查。

该策略的核心思想在于：将问题从“自动发现终止性证明”转化为“自动验证一个给定的证明”。从计算理论角度看，这种转变具有深刻意义。不可判定性结论主要限制的是自动发现算法是否存在，而在多数形式化体系中，验证一个已给出的证明往往是可判定、甚至可以在多项式时间内完成的。因此，人工注解与交互式证明并不试图突破不可判定性边界，而是通过引入人类智慧来绕开这一边界。

在具体实践中，这一思想通常体现为契约式编程与形式化规范的使用。开发者需要为程序提供前置条件（pre-condition）、后置条件（post-condition）以及循环不变式和终止性度量，而验证工具则负责检查这些条件在语义上是否成立。例如，终止性证明往往依赖于一个良序的变元函数，其值在每次迭代中严格下降且有下界，从而排除无限执行的可能性。

下面的示例以一种接近注解风格的伪代码展示了这一思想。代码中的注释并非普通文档说明，而是作为形式化验证的输入，用于明确表达终止性证明所需的信息：

```

1  int sum_to_n(int n){
2      // requires: n >= 0
3      // ranking function: R = n (non-negative, decreases by 1 each
        iteration)
4      int s = 0;
5      while(n > 0){

```

```

6      s += n;
7      n--; // R decreases
8  }
9  return s;
10 }
```

在该示例中，开发者显式声明 $R = n$ 作为变元函数，并说明其非负性与单调递减性质。验证工具只需检查这些断言在循环体中是否保持成立，即可机械化地确认程序必然终止，而无需自行“猜测”终止性证明。

在真实工程中，这类思想被广泛应用于多种形式化工具和方法中，例如基于 Hoare 逻辑的程序验证、契约式编程语言以及交互式证明助手。虽然这类方法引入了额外的开发成本，并对开发者的形式化能力提出更高要求，但它们能够在不可判定性约束下，为关键程序模块提供最强等级的可靠性保证。

从整体工程策略的角度看，人工注解与交互式证明位于终止性分析方法谱系的“最严格”一端：它牺牲自动化程度和开发便利性，换取理论上可验证、语义上可信的终止性结论。这一策略与前述的模型限制、保守近似和启发式方法共同构成了现实系统中应对终止性不可判定性的完整工具箱。

6.6 现实工具中的终止性分析实践

综合前述工程策略可以看到，现实软件系统几乎从不依赖单一的“终止性判定器”，而是采用一种多层防线（defense in depth）的组合方案。其基本思想是：用低成本、高覆盖率的方法尽早暴露明显问题，再逐步引入更精细但更昂贵的分析手段，最终由系统级机制对不可分析的情况进行兜底控制。

在实际工具链中，这种分层结构通常体现为以下几个层次：

- (1) **静态 lint / IDE 提示**：在编码阶段快速发现明显的非终止风险模式（如未更新的循环条件变量），为开发者提供即时反馈；
- (2) **静态分析与抽象解释**：在受控抽象模型下进行保守近似分析，对部分程序给出“必然终止”的可靠结论，其余情况返回“可能不终止”或“未知”；
- (3) **符号执行与约束求解**：针对特定路径、特定输入区间或关键模块进行更精细的分析，但由于路径爆炸问题，其应用范围通常受到严格限制；
- (4) **运行时监控机制**：在程序实际执行过程中监控执行时间、迭代次数或资源消耗，并在超过阈值时采取干预措施；
- (5) **系统级资源预算与隔离**：通过超时、时间片、配额或沙箱步数限制，将潜在的非终止行为约束在有限资源内。

这种组合式方案的关键优势在于：即使上层静态分析无法对某段程序的终止性给出结论，系统仍然能够通过运行时与系统级机制保证整体行为的可控性。从计算理论的角度看，这相当于在工程上将不可判定的终止性问题 PTP，逐步弱化并替换为一系列有界、近似或受限的子问题。

为了更具体地说明“有界化”如何将不可判定问题转化为可操作的工程机制，下面给出一个简化的 C 风格“步数上限执行器”（toy runtime guard）。该执行器并不尝试证明程序是否终止，而是保证：执行最多 K 步，超过该上限即强制返回失败。从理论角度看，这正对应了将一般终止性问题 PTP 替换为可判定的有界版本 BPTP。

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  typedef struct {
5      long x;
6  } State;
7
8  // 单步执行：根据当前状态更新到下一状态
9  // 返回 false 表示程序已“自然终止”；返回 true 表示继续执行
10 bool step(State *st){
11     // toy example: terminate when x reaches 0
12     if(st->x == 0) return false;
13     st->x--;
14     return true;
15 }
16
17 // 有界执行：最多执行 K 步
18 // 返回 true 表示在 K 步内终止；false 表示超出上限（未知/可能不终止）
19 bool run_bounded(State init, long K){
20     State st = init;
21     for(long i = 0; i < K; i++){
22         bool cont = step(&st);
23         if(!cont) return true; // within bound
24     }
25     return false; // timeout / step bound exceeded
26 }
27
28 int main(){
29     State init = {.x = 1000000};
30     long K = 1000;
31     if(run_bounded(init, K)){
32         printf("Terminated within bound.\n");
```



```

33     }else{
34         printf("Not terminated within bound (unknown or
           non-terminating).\n");
35     }
36     return 0;
37 }

```

该示例清楚地体现了现实系统中常见的“超时/步数上限”策略：程序是否在 K 步内终止是一个可判定的问题，而是否在无限步内终止则不可判定。工程系统通过选择合适的 K ，在安全性、性能与可用性之间取得折中。

在实际部署中，这类运行时机制往往与静态分析工具协同工作：静态分析尝试在编译期或开发期证明终止性，从而避免不必要的运行时中断；而运行时有界执行则作为最后一道防线，防止分析能力之外的行为演化为系统级风险。这种“静态证明 + 运行时兜底”的设计模式，正是不可判定性结论在工程实践中的直接体现。

7 状态机视角下的终止性分析工具链

从计算理论角度看，程序的执行过程可以统一抽象为状态转移系统，也就是 state transition system。在这一视角下，终止性分析的本质问题可以表述为：从初始状态出发的可达状态序列是否必然在有限步内到达某个终止状态，或者是否存在无限执行路径。本章将围绕这一抽象，讨论现实中常见的终止性分析工具如何在“状态机与状态空间”的框架下发挥作用。

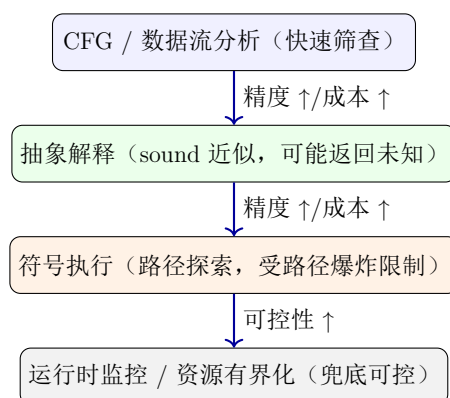


图 7: 终止性分析工具链的分层组合

7.1 控制流图与状态空间探索

控制流图 (Control Flow Graph, CFG) 是将程序结构映射为状态机的最基础工具之一，也是静态分析与程序验证中最常用的中间表示。在 CFG 中，每一个节点通常对

应一段程序位置（如基本块或语句区间），而有向边则表示在语法层面上可能发生的控制转移关系。

形式化地，可以将控制流图视为一个有向图 $G = (V, E)$ ，其中 V 表示程序点（program points）的集合， $E \subseteq V \times V$ 表示控制转移关系。CFG 的构造完全基于程序的语法结构，例如条件分支、循环和函数调用，因此不依赖于具体的输入数据或运行时状态。

为了更贴近真实的程序执行语义，通常需要在 CFG 的基础上引入状态空间的概念。具体而言，可以将程序的运行状态表示为一个二元组 (v, σ) ，其中 $v \in V$ 表示当前所处的程序位置，而 σ 表示当前的内存状态，包括变量取值、堆、栈等。在这一视角下，程序执行过程不再只是 CFG 上的节点跳转，而是一个在扩展状态空间上的路径： $(v_0, \sigma_0) \rightarrow (v_1, \sigma_1) \rightarrow (v_2, \sigma_2) \rightarrow \dots$

在终止性分析中，一个核心问题可以表述为：是否存在一条从初始状态出发的执行路径，使得状态转移序列无限延续而不进入终止状态。在 CFG 层面，这一问题往往首先体现为图中是否存在可达的环结构。特别地，循环语句和递归调用在 CFG 中必然对应某种形式的回边（back edge），从而形成潜在的无限执行路径。

下面给出一个简化的控制流与状态机示意图，展示一个典型循环结构在 CFG 中的表示方式：

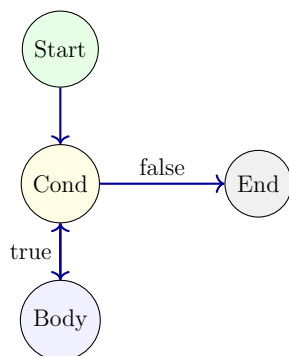


图 8: 循环结构对应的控制流状态机

在图 8 中，从 Cond 经由 Body 再返回 Cond 的路径构成一个环结构。从纯粹的 CFG 角度来看，该环是否会被执行有限次还是无限次是无法判断的，因为这一问题依赖于运行时内存状态 σ 的演化规律。换言之，CFG 只能揭示“可能存在无限执行的语法结构”，而无法区分“实际会无限执行”与“在某些条件下终止”。

因此，终止性分析往往需要在 CFG 的基础上进行进一步的状态空间探索，即同时考虑控制位置和数据状态的变化。然而，这也直接引出了状态空间爆炸问题：即使 CFG 本身规模有限，状态 σ 的取值空间也可能是无限或指数级增长的。如何在可控的成本下探索或近似这一状态空间，是终止性分析工具必须面对的核心挑战之一。

7.2 符号执行与终止性分析

符号执行（Symbolic Execution）是一种介于纯语法分析与实际运行之间的程序分析技术 [11]。它通过使用符号值而非具体数值表示程序变量，使分析工具能够在一次分析过程中同时覆盖大量可能的执行路径。在状态机视角下，符号执行可以被理解为一个符号化的状态空间中进行系统化路径探索。

在符号执行框架中，一个符号状态通常表示为 (v, φ) ，其中 v 表示当前程序位置（对应控制流图中的节点）， φ 是一个逻辑公式，用于描述当前路径上变量取值所需满足的约束条件（通常称为路径条件）。初始状态对应于程序入口位置及初始输入约束，而每一步执行都会根据程序语句对符号状态进行更新。

当程序遇到条件分支时，符号执行会沿不同分支分别生成后继状态。例如，对于条件语句 `if (x > 0)`，符号执行将产生两个符号状态：一个附加约束 $x > 0$ ，另一个附加约束 $x \leq 0$ 。这些路径条件随后交由约束求解器检查其可满足性，例如 SMT solver `cvc5`[6]，不可满足的路径将被剪枝。

在终止性分析中，符号执行的核心价值在于它能够超越具体输入，对整类执行路径进行推理。特别地，分析工具可以尝试判断是否存在一条从初始符号状态出发、满足一致约束的无限执行路径。例如，若在某个循环中，符号执行发现存在一组路径条件，使得每次迭代后这些条件依然保持可满足，并且循环条件始终为真，则该循环在语义上具有潜在的非终止风险。

从状态机角度看，这相当于在符号状态空间中发现了一条可以无限延伸的路径：

$$(v, \varphi_0) \rightarrow (v, \varphi_1) \rightarrow (v, \varphi_2) \rightarrow \dots$$

其中每个 φ_i 都是可满足的。这种分析能力使符号执行在检测数据相关的非终止行为方面，明显强于仅基于 CFG 的语法分析。

然而，符号执行同样无法回避状态空间爆炸问题。随着程序分支数目的增加，符号路径数量可能呈指数级增长；而在存在循环的情况下，符号执行往往需要对循环进行多次展开，进一步加剧路径数量的膨胀。因此，实际工具中通常会引入多种限制策略，例如限制循环展开次数、合并相似路径、或对路径条件进行抽象近似。

从计算理论视角看，符号执行并没有突破终止性不可判定的根本界限。若允许无限展开并对所有符号路径进行精确分析，符号执行本质上仍然是在尝试解决一个与停机问题等价的任務。因此，现实中的符号执行工具只能在受限条件下工作，将终止性分析转化为一系列有限、近似或启发式的探索任务。尽管如此，在关键路径分析、漏洞发现和复杂条件依赖的终止性风险识别方面，符号执行依然是一种极具价值的分析手段。

7.3 抽象解释与不动点计算

抽象解释（Abstract Interpretation）提供了一种在数学上系统化的终止性分析框架[3]。其核心思想是：在一个抽象状态空间中，对程序状态集合进行不动点计算，从而近似刻画程序的所有可能行为。为保证不动点计算在有限步内收敛，实际分析常引入 widening 与 narrowing，在牺牲精度与恢复精度之间做平衡。

设具体状态空间为 S ，抽象状态空间为 $\hat{S}$ ，并存在抽象函数 $\alpha: \mathcal{P}(S) \rightarrow \hat{S}$ ，以及相应的具体化函数 $\gamma: \hat{S} \rightarrow \mathcal{P}(S)$ 。程序的语义可以表示为一个状态转移函数 $F: \mathcal{P}(S) \rightarrow \mathcal{P}(S)$ ，而抽象语义则为 $\hat{F}: \hat{S} \rightarrow \hat{S}$ 。

终止性分析通常关注是否存在无限递增的状态序列。通过在抽象域中计算不动点 $\hat{X} = \hat{F}(\hat{X})$ ，分析器可以判断某些循环是否“稳定”在某个抽象状态，从而推断其可能的非终止行为。

在状态机视角下，抽象解释相当于对原始状态机进行“压缩”，把大量具体状态合并为有限个抽象状态，使状态空间变得可遍历。

7.4 运行时检测与监控机制

当静态分析或符号化方法无法对程序终止性给出明确结论时，运行时检测（Runtime Monitoring）成为终止性分析的重要补充手段。与前述方法不同，运行时监控并不试图分析程序的所有可能行为，而是仅沿着实际执行路径对状态演化进行观察和干预。

从状态机的角度看，运行时监控相当于在程序执行过程中动态跟踪当前状态 (v, σ) 的变化，并依据预先设定的规则判断执行是否仍在“合理进展”。与静态方法相比，其优势在于能够直接利用真实运行时信息，而不需要对所有潜在路径进行推理。

一种常见的监控策略是检测“进展缺失”或“状态重复”。例如，系统可以记录某些关键状态变量（如循环计数器、任务队列长度或资源使用量）的变化趋势；若在多次迭代中这些变量未发生实质变化，或者执行反复经过相同的状态模式，则可以推断程序可能陷入无限循环或活锁状态。这类检测并不试图严格证明非终止性，而是作为一种风险信号触发后续处理。

在工程实践中，运行时监控往往与资源约束机制紧密结合。系统通常会对执行步数、运行时间、内存或能耗设定明确上界，一旦超过阈值便强制中断执行或回滚状态。从形式上看，这相当于在原有状态机中人为引入一个“超时终止状态”，无论程序本身是否会自然终止，系统都保证执行过程在有限时间内结束。

这种策略并不违背终止性不可判定的理论结论：它并未声称判断程序是否会终止，而只是通过外部约束将不可控的执行行为转化为可控的、有界的系统行为。因此，运行时检测与监控机制在终止性分析工具链中扮演着“最后防线”的角色，与静态分析和形式化方法形成互补。

7.5 Trace、Profiler 与终止性定位

Trace 与 Profiler 工具并不直接解决终止性判定问题，但在定位终止性相关缺陷时发挥着关键作用。从状态机角度看，Trace 可以被视为程序实际执行路径在状态空间中的一条具体轨迹。

设执行过程中记录的状态序列为 $(v_0, \sigma_0), (v_1, \sigma_1), \dots, (v_n, \sigma_n)$ 。通过分析该序列中是否存在周期性模式，如同一程序点反复出现且状态变化有限。工程师可以推断程序是否陷入某种“局部循环”。

Profiler 则从统计角度揭示状态访问频率：若某些状态或边被访问的次数异常高，则往往意味着潜在的非终止或性能退化问题。这类工具将“是否终止”这一布尔问题，转化为“哪些状态被过度访问”的诊断问题。

在工程实践中，Trace 与 Profiler 通常对应两种互补的观测方式：

- **Trace（详细轨迹）**：通过插桩记录“发生了什么”，例如函数调用序列、关键变量变化、循环迭代次数、锁的获取与释放等。它能帮助回答“卡死前最后在做什么、是否在某个循环/等待点附近震荡”。代价是开销较大、数据量大，适合复现后对局部片段做精细观察。
- **Profiler（统计采样）**：通过采样或计数记录“时间主要花在哪里”，例如采样调用栈、统计热点函数与热点基本块。它不依赖完整轨迹，也更容易在生产环境使用，适合快速定位“最可疑的热点回路/等待点”。

从状态机视角看，这两类工具分别对应“取一条路径看清细节”与“对大量路径片段做统计聚合”。当终止性风险表现为死循环或活锁时，常见现象是：Trace 中出现高度重复的局部模式（同一控制点反复出现）；Profiler 中出现某个循环体、某个自旋等待点或某个锁竞争路径长期占据 CPU。相反，当问题更像死锁或外部依赖阻塞时，Profiler 可能显示线程长时间停留在等待/阻塞调用，而 Trace（或线程栈快照）会暴露等待关系与资源占用链条。

要点：把“终止性问题”先改写成“可定位的问题”】【工程视角】

对工程排障而言，第一目标通常不是证明“永不终止”，而是回答更可操作的问题：卡在哪里、循环变量是否进展、是否在等待外部资源、是否存在热点回路。Trace/Profiler 正是把抽象的终止性风险，改写为可观测、可归因、可修复的诊断任务。

7.6 个人思考：状态空间爆炸与分析边界

从状态机视角统一考察上述工具，可以发现它们都在以不同方式应对同一个根本挑战：状态空间爆炸。无论是符号执行中指数增长的路径数量，还是抽象解释中抽象精度与计算成本之间的权衡，其背后都体现了通用计算模型的复杂性。

不可判定性从理论上说明：不存在一种对所有程序都完备、自动且高效的终止性分析方法。状态机视角进一步揭示了这一限制的结构性的原因在于，程序的状态空间在一般情形下是无限且高度分支的。

因此，现实工具链的成功并不在于“战胜不可判定性”，而在于在理论边界内进行合理取舍：通过状态抽象、路径剪枝、有界探索和运行时监控，将不可控的无限状态空间转化为工程上可管理的近似模型。这种思想，正是计算理论在现实系统中发挥价值的典型体现。

8 一个可执行的简化模型：终止性检测的模拟实现

在前述章节中，本文从理论层面证明了一般意义下终止性检测问题的不可判定性，并分析了现实工程实践中常见的应对策略。本章进一步构建一个可执行的简化模型，用于直观展示不同终止性检测策略在实际运行中的行为差异。该模型并不试图逼近真实编程语言或工业级分析工具，而是作为一个教学性与解释性的实例，用以连接计算理论结论与工程实践取舍。

8.1 设计目标与建模假设

本模型的设计目标并非复现真实编程语言或工业级终止性分析工具，而是构建一个足够简单、但结构完整的可执行系统，用于直观展示不同终止性检测策略在同一语义框架下的行为差异。具体而言，模型的设计目标包括以下三点：

- (1) 以最小化的语言结构呈现程序执行的状态机本质，使终止性问题能够被明确表述为“是否存在无限执行路径”；
- (2) 在统一的执行语义下实现多种终止性检测策略，避免因模型差异掩盖策略本身的优缺点；
- (3) 通过实际可运行的解释器示例，直观体现不可判定性结论在工程实现中的具体影响。

为达成上述目标，我设计模型时作出了一系列刻意的简化假设。首先，程序仅操作整数变量，不包含复杂数据结构或指针操作，从而避免内存别名等额外复杂性。其次，

控制结构仅包含顺序执行、条件分支与循环，不支持递归与函数调用，以保证状态机结构清晰可控。此外，模型不考虑并发、异常处理或外部 I/O，这使得程序行为完全由内部状态决定。

这些假设显著削弱了语言的表达能力，但它们并不影响终止性问题的核心结构：即循环与状态演化是否存在无限延续的可能。因此，该模型在牺牲工程复杂度的同时，仍然保留了分析终止性所需的关键要素，适合作为理论结论与工程实践之间的桥梁。

8.2 简化程序语言与执行语义

为了构建一个可执行且易于分析的终止性研究模型，本文引入一种极简的命令式程序语言，记为 $\mathcal{L}_{toy}$ 。该语言刻意舍弃复杂的数据结构与控制机制，仅保留终止性分析所需的最基本要素，其语法定义如下：

$$\text{stmt} ::= x := c \mid x := x + c \mid \text{if } x > 0 \text{ then } S_1 \text{ else } S_2 \mid \text{while } x > 0 \text{ do } S$$

其中， x 表示整数变量， c 为整数常量， S 表示语句序列。该语言包含赋值、条件分支与循环三类核心控制结构，已经足以表达有限执行、条件终止以及潜在非终止等多种典型程序行为。

为避免概念被“多线程/非确定性”分散注意力，本文默认 $\mathcal{L}_{toy}$ 的执行是确定性的：给定当前配置 (pc, σ) 后，下一步配置 (pc', σ') 是唯一确定的。这样，终止性就可以被直观地理解为：从初始配置出发的那条唯一执行路径是否会在有限步内到达终止配置。若该路径无限延伸，则对应“不终止”。

下面给出两个极简片段，展示 toy 语义下“终止/不终止”的差异：

```

1 // Terminates: x decreases and will reach 0
2 while (x > 0) do
3     x := x - 1;
4
5 // Diverges: x increases so the guard stays true
6 while (x > 0) do
7     x := x + 1;

```

Listing 1: Toy 示例：单调下降（终止）vs 单调上升（不终止）

在第一个例子中，存在显式的“下降度量”（变量 x 在自然数上严格下降且有下界），因此终止性容易证明；而在第二个例子中，循环条件始终保持为真，配置序列会无限延伸，从而产生非终止行为。现实中的终止性证明与工具提示，往往就试图在更复杂的程序结构中寻找类似“下降趋势/边界约束”的证据。

要点：toy 模型里终止性就是“路径是否有限”】【与状态机视角统一】

在确定性 toy 语义下，终止性等价于从初始配置出发的状态机路径是否为有限长度。这样，终止性讨论可以自然地转化为对配置序列、循环回边与度量函数的讨论，为后文三类检测策略提供统一参照。

为了形式化刻画程序的运行过程，本文将程序状态定义为一个二元组 (pc, σ) ，其中 pc (program counter) 表示当前即将执行的程序位置， σ 表示变量到整数值的映射，即当前的内存环境。直观地说， pc 决定“执行到哪一条语句”，而 σ 决定“该语句在什么数据环境下执行”。

在该模型中，程序执行语义被抽象为状态之间的转移关系 $(pc, \sigma) \longrightarrow (pc', \sigma')$ ，该关系由当前语句的类型唯一确定。例如，赋值语句会更新变量映射 σ 并将 pc 移动到下一条语句；条件语句根据 $\sigma(x)$ 的取值选择不同的后继程序位置；循环语句则可能将控制流重新导向循环起点。

当程序计数器 pc 到达程序末尾位置时，程序不再存在后继状态，此时认为程序自然终止。

8.3 解释器的状态机实现

基于上述语义定义，可以将程序的解释执行过程本身视为一个状态机。该状态机的每一个状态对应一个程序执行状态 (pc, σ) ，而状态之间的转移正对应于解释器执行一条语句的过程。

从这一视角看，解释器并不“理解程序整体是否终止”，而只是机械地根据当前状态和语句语义，不断计算下一个状态。这一过程在终止性分析中具有关键意义：程序是否终止，等价于该状态机是否存在一条从初始状态出发、在有限步内到达终止状态的执行路径。

图 9 展示了简化解释器的整体状态机结构。图中从“取指/译码”到“执行语义更新”，再到“终止条件检查与控制点更新”，构成了解释器的一次完整执行循环。若终止条件未满足，状态机会回到取指阶段继续执行；若满足，则进入终止状态。

需要注意的是，该状态机本身并不保证一定终止。若程序语义导致 pc 与 σ 的组合不断重复或无界变化，则解释器将沿着一条无限路径持续运行。这一现象正是程序终止性问题在解释器层面的直接体现，也为后续不同终止性检测策略的对比实验提供了统一的执行基础。该状态机本身可能产生无限路径，这正是终止性问题在解释器层面的直接体现。

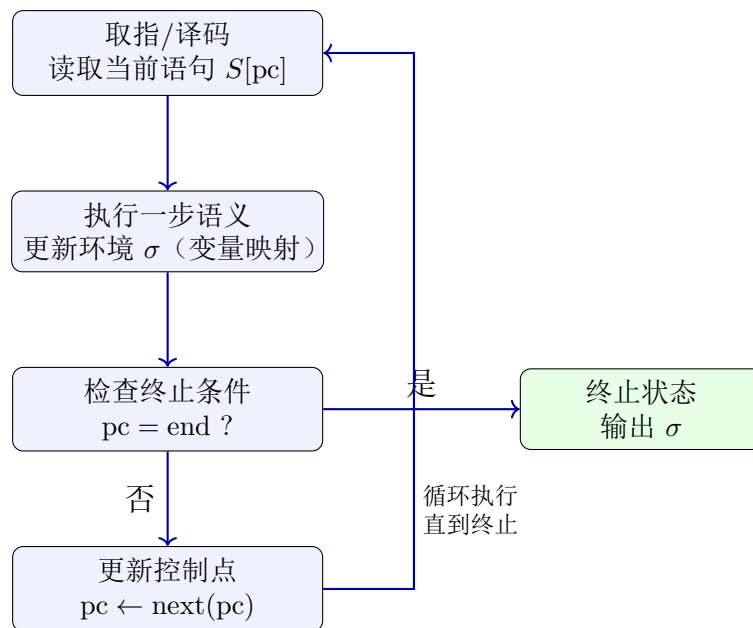


图 9: 简化解释器的状态机结构：取指、执行、判定、更新、循环

8.4 三类终止性检测策略的实现

在上述统一的解释器状态机框架下，本文实现并对比三种具有代表性的终止性检测策略。它们在设计目标、可保证性质以及工程代价上各不相同，分别对应理论上“可识别但不可判定”“有界可判定”以及“启发式近似分析”等不同思想。

策略	自身是否必停机	输出结论强度	典型工程含义
完全模拟执行	不保证	终止时给出 YES; 不终止时可能无输出	等价于“跑一跑看看”，适合小规模/可控输入
有界步数执行	保证	YES 强; NO 弱 (仅表示未在界限内终止)	超时/步数限制的形式化版本，可控但不完备
规则静态检测	保证	结论依赖规则覆盖; 通常返回 YES/UNKNOWN	低成本筛查，适合早期提示与审查

表 2: 三类终止性检测策略的性质对比（概述）

要点：三类策略分别“退让”了什么

完全模拟不退让语义，但退让了“自身必停机”；有界执行退让了“能否区分不终止”，换取可控预算；规则检测退让了覆盖与精确性，换取极低成本与即时反馈。三者对应的并非三种“更好或更差”的算法，而是三种面向不同工程目标的取舍。

8.4.1 完全模拟执行

完全模拟执行策略尝试忠实地按照程序语义运行解释器，直到程序自然终止为止。这一策略不引入任何额外假设，也不对执行过程施加限制，因此在语义上最为直接。

```
1 bool run_full(State st){
2     while(true){
3         if(is_terminated(st)) return true;
4         st = step(st); // may loop forever
5     }
6 }
```

从理论角度看，该策略在程序确实终止时一定能够观察到终止状态，并给出正确结论；但若程序本身不终止，解释器也将沿着无限执行路径持续运行，无法返回结果。因此，这一策略只能“识别”终止的程序实例，却无法对所有输入保证自身停机。

这种行为与终止性问题的理论性质完全一致：终止性集合是图灵可识别的，但不可判定。完全模拟执行正是这一理论结论在实现层面的直接体现。

8.4.2 有界步数执行

有界步数执行策略在完全模拟的基础上，引入一个显式的执行步数上界 K ，将终止性检测问题转化为“是否在 K 步内终止”的有界判定问题。

```
1 bool run_bounded(State st, int K){
2     for(int i = 0; i < K; i++){
3         if(is_terminated(st)) return true;
4         st = step(st);
5     }
6     return false; // timeout
7 }
```

由于执行步数被严格限制，该策略保证自身必然停机，因此在工程上具有良好的可控性。当函数返回 `true` 时，可以确信程序在给定步数上界内终止；而当返回 `false` 时，结论仅表示“在当前界限内未观察到终止”，并不能区分程序是否真的不终止。

从计算理论视角看，这一策略对应于将不可判定的终止性问题 PTP 转化为可判定的有界版本 BPTP。这也解释了现实系统中超时机制、步数限制等设计的理论合理性。

8.4.3 基于规则的静态检测

第三种策略在程序执行前进行简单的静态结构分析，尝试通过局部规则推断程序是否“很可能”终止。例如，若循环条件依赖的变量在循环体中被单调更新，则可推断该

循环存在终止趋势。

```
1 bool rule_based_check(Program p){
2     if(loop_condition_uses_x(p) &&
3         loop_body_modifies_x(p)){
4         return true; // likely terminates
5     }
6     return false; // unknown
7 }
```

该策略不模拟程序执行，也不构建完整状态空间，而是依赖经验性或语法层面的模式匹配，因此计算成本极低、响应迅速。然而，其结论本质上是启发式的：规则成立时通常意味着程序终止，但规则不成立并不等价于程序不终止。

因此，基于规则的静态检测在终止性分析中通常是不完备甚至不完全可靠的，但在实践中却具有较高性价比，尤其适用于 IDE 提示或代码审查的早期筛查阶段。

8.5 结果与案例分析

为比较三种终止性检测策略在 toy 语义下的行为差异，本文选取若干具有代表性的程序片段进行对比分析。样例覆盖：易于判定终止的简单循环、结构上更复杂但仍会终止的循环，以及在模型语义下确实不终止的典型错误模式。

案例一：简单倒数计数循环 该程序使用单调递减变量作为循环控制条件，符合典型的可验证终止模式。

```
1 int case1(int x){
2     // assume x >= 0
3     while(x > 0){
4         x = x - 1;
5     }
6     return 0;
7 }
```

Listing 2: 案例一：倒数计数循环（必然终止）

输入 x	案例一：完全模拟步数	案例一：有界模拟所需 K
10	10	≥ 10
100	100	≥ 100
1000	1000	≥ 1000

表 3: 案例一在不同输入规模下的步数需求

对该程序而言，三种策略都能给出一致结论：完全模拟在 x 步内终止；有界策略在 $K \geq x$ 时返回 YES；规则检测可直接识别出控制变量单调递减的结构。

案例二：条件分支导致的非线性下降 该程序在循环体中包含条件分支，使得变量变化并非线性或单调，但在实际运行中仍会收敛至终止状态。为保持在 $\mathcal{L}_{toy}$ 的表达范围内，这里用“偶数减半、奇数减一”的结构来制造非线性但可终止的行为（相较线性递减更难被简单规则捕捉）。

```

1  int case2(int x){
2      // assume x >= 0
3      while(x > 0){
4          if(x % 2 == 0){
5              x = x / 2;
6          }else{
7              x = x - 1;
8          }
9      }
10     return 0;
11 }

```

Listing 3: 案例二：分支导致的非线性下降（实际终止，但规则可能无法证明）

该程序在 toy 语义下仍然会终止，但“下降趋势”不再是简单单调递减，因此基于局部模式的规则检测通常会返回未知；而完全模拟与有界模拟可以通过执行语义确认终止。对输入规模较大的情况，有界策略需要选择足够的 K 才能观察到终止，但这一做法换来了“分析过程必然停机”的可控性。

输入 x	案例二：完全模拟（大致步数）	有界策略建议 K
10	≈ 8	50
10^3	≈ 25	200
10^6	≈ 45	500

表 4: 案例二的步数量级与有界参数选择（toy 语义下的量级估计）

案例三：潜在非终止程序 为了构造模型语义下确实可能无限执行的程序，可采用“循环条件依赖变量，但循环体不改变该变量”的典型错误模式。

```

1  int case3(int x){
2      // if x > 0 initially, loop never terminates
3      while(x > 0){
4          // BUG: x never changes

```

```

5      // do something irrelevant
6  }
7  return 0;
8  }

```

Listing 4: 案例三：潜在非终止（循环条件变量不更新）

对于该程序，完全模拟执行在实验中表现为解释器自身无法停机；有界执行在达到步数上界后返回超时结果；规则检测通常能够直接标记其为“高度疑似不终止”或返回“未知/不安全”。该行为与前述不可判定性讨论并不矛盾：这里的非终止属于易被语法规则捕捉的特例。

策略	案例一（倒计时）	案例二（非线性下降）	案例三（不更新变量）
完全模拟（full）	终止	终止	可能不返回
有界模拟（bounded, K ）	YES ($K \geq x$)	YES (K 足够大)	NO_WITHIN_K
规则检测（rule-based）	likely	unknown	likely non-terminating

表 5: 三类策略的输出对比（基于 toy 语义的推导结论）

综合上述实验可以观察到，三种策略在行为上的差异具有高度一致的模式：完全模拟执行在终止性判断上最精确，但缺乏可控性；有界执行始终保证自身停机，但只能提供有限结论；规则检测计算成本最低，却只能覆盖有限的程序模式。这些结果并非实现缺陷，而是终止性不可判定性在工程层面的直接体现。

8.6 个人思考：工程工具的设计取舍

通过该可执行简化模型的构建与实验分析，可以更直观地理解终止性检测工具在工程设计中所面临的根本取舍。终止性检测并非一个单纯的算法问题，而是围绕三项核心因素展开的系统性权衡：程序表达能力、分析精度以及执行成本。

完全模拟执行策略在语义上最为直接，能够在程序实际终止时给出确定结论，但其自身并不保证停机，因此在工程环境中难以作为独立方案使用。有界执行通过引入外部限制，主动放弃对“无限行为”的区分能力，从而换取分析过程的可控性；而基于规则的静态检测则进一步牺牲普适性，以极低成本覆盖大量常见错误模式。

该模型清楚地表明，不存在一种“单一策略”能够在所有维度上占优。现实工具链之所以采用多策略组合，正是因为不同方法在不同维度上具有互补优势：规则检测用于快速筛查明显问题，有界或近似分析覆盖更复杂的情况，而运行时机制则作为最后防线，防止理论上无法判定的行为演化为系统级风险。

另一个容易被忽视的工程因素是“误报的可管理性”。在大规模代码库中，过高的误报会迅速消耗开发者对工具的信任，最终导致规则被整体关闭。实践中常见的做法是给告警设置分级阈值，或以“潜在风险分数”的方式排序，让最危险的模式优先进入人

工复核；同时用运行时监控的真实超时/异常数据反哺静态规则，逐步缩小规则的误报范围。这种“静态筛查 + 运行时证据”的闭环，是让终止性分析真正落地的关键。

从计算理论视角看，不可判定性并未阻止终止性分析工具的发展，而是为其合理定位提供了清晰边界。一个成功的工程工具，并不试图突破不可判定性结论，而是在这一边界内，通过模型限制、近似分析与系统约束的组合，构建出在实践中足够可靠、可控且高效的解决方案。近几年这类工作也常以工具与竞赛的形式持续推进。例如 FSCD 2023 的终止性方向综述与竞赛讨论 [12]。TACAS 的竞赛工具贡献覆盖不同思路：AProVE 给出 C 程序非终止见证 [7]，VeriFuzz 面向（非）终止性检查 [8]，以及基于抽象解释的 Goblint[9] 与 PROTON[10] 等。

9 个人综合思考与扩展应用

在前文中，本文从计算理论的角度系统讨论了程序终止性检测问题的形式化建模、不可判定性证明以及现实工程中的应对策略。本章在此基础上，从个人理解出发，对这些理论结论进行进一步反思，并尝试回答一个更偏“方法论”的问题：当我们已经知道某些分析目标在理论上不可实现时，计算理论究竟还能为工程实践提供什么价值？

本章依次讨论三件事：计算理论如何帮助理解程序分析的极限；终止性检测里正确性、完备性与可用性的结构性权衡；以及若干现实系统场景中，不可判定性结论如何影响工程设计决策。

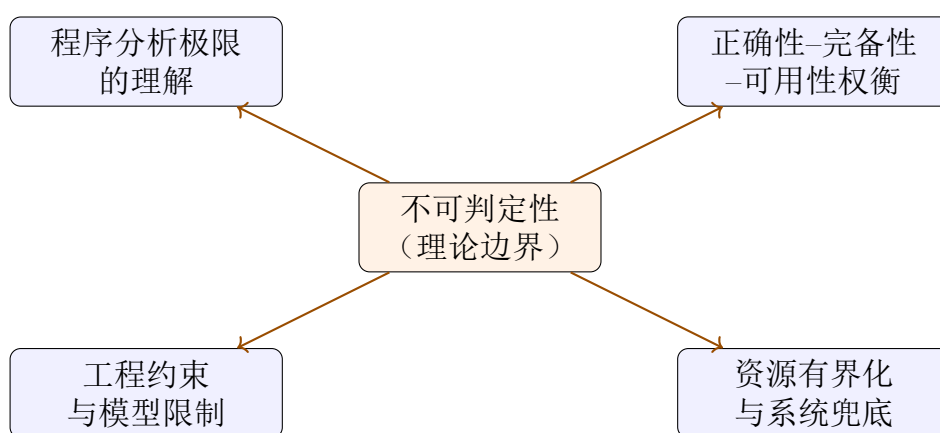


图 10: 本章讨论思路：不可判定性如何影响程序分析与工程设计

9.1 计算理论对理解程序分析极限的价值

计算理论最重要的价值之一，在于为程序分析问题提供清晰而不可逾越的理论边界。在缺乏相关理论背景时，人们往往倾向于将程序分析工具的不足归因于“算法还不够复杂”或“工程实现尚待改进”，从而不断尝试以更复杂的规则或更强的计算资源来弥补分析能力的不足。

然而，停机问题与 Rice 定理所揭示的不可判定性表明：即使忽略所有工程实现细节，某些分析目标在通用计算模型下本身就不存在完全自动的解决方案。终止性检测问题正是这一思想的集中体现。一旦程序具备图灵完备的表达能力，其运行行为便可以嵌入任意复杂的计算过程，此时试图通过统一算法对所有程序的终止性给出绝对正确的判断，必然会触及不可判定性边界。

因此，计算理论在这里并不是提供“更强的分析工具”，而是帮助我们识别哪些目标在理论上是不现实的。这种认知具有重要的实践意义：它促使研究者与工程师在设计分析工具时，主动明确工具的适用范围、保证条件与失败模式，而不是将分析失败视为偶然或技术债务。

从这一角度看，计算理论更像是一种“约束性知识”：它并不直接告诉我们该如何实现一个工具，而是告诉我们哪些方向值得努力，哪些方向注定徒劳。

9.2 终止性检测中的正确性、完备性与可用性权衡

在终止性检测的工程实践中，分析方法往往需要在多个目标之间进行权衡，其中最核心的是正确性（soundness）、完备性（completeness）与可用性（usability）之间的关系。这三者在通用程序模型下无法同时满足，而不可判定性正是这一冲突的理论根源。

更形式化地说，假设存在一个分析器 A ，对任意 $\langle P, x \rangle$ 都能在有限时间内输出 YES/NO，并且同时满足：YES 时一定终止（正确性），终止时一定输出 YES（完备性）。那么 A 将直接成为 PTP 的判定器，从而与前文证明的不可判定性矛盾。工程上所谓的“三者权衡”，本质上正是对这一逻辑不可能性的不同“退让方式”。要么放弃完备性（返回未知/保守），要么放弃绝对正确性（启发式误差），要么通过限制模型与有界化把问题改写为可判定版本。

要点：把 soundness/completeness 说清楚

在终止性分析语境下，一个常见做法是允许分析器输出三值结果 {YES, NO, UNKNOWN}：

- **可靠性 (soundness)**：若输出 YES，则程序确实终止（不把不终止误判为终止）；
- **完备性 (completeness)**：若程序确实终止，则分析器能输出 YES（不会把终止程序永远卡在 UNKNOWN/超时）；
- **可用性 (usability)**：在合理时间/资源内给出结论的比例与代价（即使是 UNKNOWN 也要“及时”）。

不可判定性迫使我们在三者之间做取舍：想要强可靠性，往往就要接受更多 UNKNOWN；想要更强覆盖与自动化，往往就要接受启发式误差或资源上界。

强调正确性的分析方法通常采用保守近似或形式化证明思路。这类方法的优势在于：一旦给出“程序终止”的结论，便可以在理论上加以保证；其代价则是覆盖范围有限，且往往需要额外的人工注解或严格的程序结构约束。因此，这类方法更适合安全关键系统或高可信软件，而不适合快速迭代的通用开发场景。

相比之下，强调可用性的工具更倾向于启发式规则与经验分析。它们可以快速覆盖大量常见错误模式，在日常开发流程中具有很高的性价比，但不可避免地会产生误报或漏报。从计算理论角度看，这种不完备甚至不完全可靠的行为，并非设计缺陷，而是不可判定性约束下的自然结果。

在实际系统中，许多成熟工具链并不会试图在这三者中“选一个最优点”，而是通过多层次策略组合来实现整体平衡：静态规则用于快速筛查，近似分析用于覆盖复杂结构，运行时机制作为兜底手段。这种分层设计正是对理论边界的理性回应。

9.3 自己设计的应用场景与工程启示

基于上述理论分析，下面结合个人理解，讨论若干现实系统中终止性问题的具体应用场景，说明计算理论如何在工程决策中发挥指导作用。

9.3.1 代码审查与自动化质量控制

在代码审查与持续集成环境中，终止性检测的主要目标并非严格证明程序一定终止，而是尽早暴露明显且高风险的非终止模式。例如，循环条件依赖的变量在循环体中未发生变化，或递归函数缺乏规模缩减参数，都是实践中频繁出现的缺陷来源。

在这一场景下，基于规则和启发式的静态分析工具具有较高实用价值。尽管这些规则无法覆盖所有可能的非终止行为，但它们能够以较低成本显著提升代码质量。从理论角度看，这种做法等价于主动放弃完备性，以换取分析效率和工程可落地性，是不可判定性约束下的理性选择。

更具体地，代码审查中常见的“终止性检查清单”往往聚焦于能被人类快速验证的结构性条件：

- **循环控制变量**：循环条件依赖的变量是否在循环体内被更新，且是否存在明显的单调趋势或边界保护；
- **退出条件与防护**：是否存在 `break/return` 等逃逸路径，是否有最大迭代次数、超时或熔断等兜底；
- **递归规模下降**：递归调用的参数是否严格“变小”，是否有明确的基例（base case）；
- **外部依赖**：网络/锁/队列等调用是否设置超时，避免“外部不返回”演化为整体不终止。

这类检查并不宣称“证明终止”，但能把大量工程事故源头提前暴露出来，属于在不可判定边界内仍然性价比较高的一类治理手段。

9.3.2 安全关键系统中的可验证编程子集

对于航空航天、工业控制等安全关键系统而言，程序终止性往往是系统正确性的基本前提。在此类场景中，与其试图对通用程序进行终止性判定，不如从源头上限制程序模型，使其天然具备可验证性。

具体而言，可以通过限制语言特性、禁止无界循环和递归、要求显式循环上界或形式化变元函数等方式，将终止性问题转化为在受限模型下的可判定问题。这种策略体现了计算理论的重要启示：当问题在通用模型下不可判定时，可以通过改变模型假设来获得可分析性，而不是在原模型中徒劳追求完美算法。

在“可验证子集”中，终止性证据往往以更工程化的方式出现：循环上界来自协议/控制周期的物理约束，或来自经过审计的常量；递归被结构化为受控的迭代；复杂控制流被重写为更易分析的结构。对于需要自动化的部分，则可以借助规模下降原则等结构化判据来生成或检查终止证据 [5]。这种路径并不追求在所有程序上“自动判定”，而是把系统约束与证明义务写进设计，从而让关键性质变得可审核、可复用。

9.3.3 在线服务中的资源预算与有界执行

在在线服务和后端系统中，终止性问题往往以资源耗尽或服务阻塞的形式出现。系统设计者通常并不关心程序是否在理论上终止，而更关心其是否在可接受的时间和资源

范围内完成执行。

因此，引入超时机制、步数限制或资源配额成为常见做法。从形式化角度看，这相当于将终止性检测问题从 PTP 转化为有界版本 BPTP，后者在理论上是可判定的。这种设计并非“规避理论问题”，而是通过重新定义问题目标，使系统行为在工程上保持可预测和可恢复。

在工程实现上，资源预算常以“多维度上界”出现：请求整体超时（wall-clock），单次下游调用超时（network/IO），并发与队列长度上界（避免排队导致的“看似不终止”），以及 CPU/内存配额（避免资源挤占）。当预算耗尽时，系统的目标不是给出“程序不终止”的语义结论，而是触发降级或失败返回，并把诊断信息（输入、路径、指标）留给后续排查。换句话说，在线服务关心的是把无界风险变成可观测、可恢复的事件。

9.3.4 智能合约与脚本沙箱的步数限制

在智能合约平台与脚本沙箱系统中，非终止行为可能带来直接的经济损失或安全风险。因此，这类系统普遍采用 Gas 机制或解释器步数限制，对每一次执行施加严格的资源上界 [4]。

从计算理论视角看，该机制并不是终止性判定算法，而是一种强制有界执行策略。它通过制度化地限制状态空间探索深度，避免不可判定性导致的系统失控。这一做法清楚地表明：理论结论并非停留在抽象层面，而是直接塑造了现实系统的执行模型与安全机制。

同时，Gas/步数限制也带来一个很重要的“可计费/可预测”性质：执行成本变为显式资源，开发者必须在合约或脚本设计时考虑最坏情况下的资源消耗，从而在语言层面形成一种“以资源为中心的可用性约束”。这再次体现了边界对工程设计的反向塑形：当无法判定终止性时，就把目标改写为“在预算内要么完成，要么可控失败”。

9.3.5 批处理任务与数据管道的终止性治理

在离线数据处理与批任务调度中，“不终止”往往表现为任务长期占用资源、下游任务被阻塞或重试风暴。工程上常用的治理方式包括：为每个任务设置重试上限与总执行时长预算；将任务拆成可检查点的阶段，失败后从最近检查点恢复；对输入规模建立配额或分片策略，避免单个任务吞噬全局资源。这些机制并不解决理论上的终止性问题，但能把“不可预测的无限执行”转化为“可恢复的超时失败”，从而确保系统整体吞吐与稳定性。

10 结论

本文围绕“程序自动终止性检测问题”，从建模、不可判定性证明到工程应对做了一次贯通。文章先从现实系统里常见的“卡死/跑不完”现象出发，把问题统一成形式语言框架下的判定问题；再在状态机视角下引入图灵机，并用归约、反证法与 Rice 定理刻画终止性检测的理论边界；最后回到工程实践，说明现实系统为何必须用多策略、分层防线来管理终止性风险，并通过一个可执行的简化模型对比不同策略的行为差异。通过这条往返路径，本文想强调的是：计算理论不只是给出“能或不能”的答案，更重要的是帮我们把边界说清楚，从而让工具与系统的设计承诺更可信。

10.1 程序自动终止性检测问题的理论总结

从形式化角度看，程序自动终止性检测问题可以被定义为判定集合

$$PTP = \{\langle P, x \rangle \mid P \text{ 在输入 } x \text{ 上最终终止}\},$$

其本质是判断程序执行对应的状态转移序列是否在有限步内到达终止状态。这个建模的关键在于：一旦把现实程序的执行过程抽象为状态机（更精确地说是配置转移系统），终止性就转化为“是否存在无限路径”的性质；再把程序统一映射到图灵机模型后，这一性质与停机问题在结构上同构，从而可以直接使用计算理论的既有结论来刻画其极限。

在理论层面，本文通过构造从停机问题 $HALT_{TM}$ 到终止性判定问题 PTP 的多对一归约，给出了标准的不可判定性证明：若存在对任意 $\langle P, x \rangle$ 都保证停机且完全正确的判定器，则可以用它来判定任意 $\langle M, w \rangle$ 是否停机，从而与停机问题不可判定性矛盾。这一结果明确表明：“通用、全自动、完备正确并保证自身停机”的终止性检测器不存在。这不是某个分析方法做得不够好，而是通用计算模型下不可绕开的理论边界。

进一步地，本文从 Rice 定理视角解释了终止性不可判定的普遍性：终止性属于依赖程序运行语义的性质，而非单纯的语法结构性质；只要性质是非平凡的语义性质，也就是对某些程序成立、对某些程序不成立，就不可判定。换言之，终止性问题并非“特例”，它代表了语义分析中一大类不可判定问题的典型形态。这一点对于理解程序分析领域“为何总会有分析盲区”尤为重要：盲区并非来自实现细节，而来自语义性质的本质复杂性。

另外还需要分清一个实践上很关键的点：不可判定性否定的是无界、通用、完备的判定；而受限模型、有限边界与“可验证但不完备”的策略仍然可以提供稳定且可落地的工程价值，见图2所示的工程场景。

10.2 不可判定性对现实软件系统设计的启示

不可判定性结论并不意味着终止性分析在工程实践中毫无意义，相反，它为现实系统设计提供了明确的指导原则：不可判定性为分析工具的能力设定了上限，避免对“全自动、完全正确”的不现实期待。工程上真正危险的并非“工具不完美”，而是对工具能力产生错误假设：例如把启发式规则的结论当作严格保证，或误认为静态分析覆盖了所有路径。理解这一边界，反而会促使我们在设计阶段明确工具的适用范围与保证强度，从而在系统层面避免“错误确定性”带来的风险。

其次，本文梳理的工程策略表明，现实系统之所以能够“管理终止性风险”，关键在于主动放弃某些目标并引入约束，从而把不可判定问题转化为可控问题。具体而言：

- **限制程序模型**：通过语言子集、编程规范或结构化控制流约束，降低表达能力以换取可验证性；
- **保守近似与不完备**：在抽象解释等框架下优先保证可靠性 soundness，即宁可返回未知，也不轻易给出错误的“必然终止”结论；
- **启发式与经验规则**：在成本可控的情况下覆盖大量常见错误模式，用“快而不完美”的方式提供工程价值；
- **人工注解与交互式证明**：将“自动发现证明”替换为“自动验证证明”，在关键模块上获取最强保证；
- **资源预算与有界执行**：超时、时间片、配额、步数限制等机制把无界终止性问题替换为有界版本，从而让系统行为在最坏情况下仍可控。

从状态机视角来看，上述策略都可以理解为对“无限状态空间”的不同处理方式：限制模型等价于缩小状态空间；抽象解释等价于合并状态、进行压缩；符号执行等价于在约束空间中探索代表性路径；运行时监控与资源预算则等价于对状态机增加外部“截断”规则，使系统在最坏情况下也能进入某个受控终止或中止状态。现实软件系统的成功往往不在于“判断所有程序是否终止”，而在于通过体系化设计，把“可能不终止”的风险封装为可测、可控、可恢复的工程事件。

这一启示在多个真实场景中具有直接对应关系。例如，操作系统通过调度与资源隔离阻止单个任务拖垮系统；在线服务通过请求超时、熔断与限流避免无限等待；智能合约通过 Gas 机制将执行步数显式有界化；脚本沙箱通过解释器步数限制与监控将不可控执行转化为可控失败。可以说，不可判定性不仅解释了“为什么终止性检测无法完美”，也解释了“为什么这些系统机制几乎是必需的”。

10.3 局限与可能扩展

本文讨论主要集中在确定性程序与单次执行的实例终止性上，并采用简化解释器作为实验载体。这种处理有助于突出理论主线，但也带来一些边界与不足：

- **模型简化：** toy 语言省略了递归、复杂数据结构、异常与 I/O 等特性，使得“状态机本质”更清晰，但与真实语言在状态空间结构上仍有差距；
- **并发与交互：** 现实系统大量包含并发、锁、消息队列与交互式输入，此时“终止性”往往与公平性、调度与环境假设纠缠在一起，单纯的单路径模型不足以覆盖；
- **量化评测不足：** 文中对误报/漏报与性能开销主要做定性讨论，尚未在真实代码库上给出系统性的覆盖率与代价评估。

进一步的扩展方向可以沿着“增强可验证性”与“增强工程落地性”两条线推进：其一，在理论与方法上引入更结构化的终止性判据（如规模下降原则）并把它们转化为可检查的证明证书 [5]；其二，在工具链上结合抽象解释与约束求解器，对特定片段生成更精细的终止性条件，并在可行时把条件交给 SMT 求解器验证 [3, 6]；其三，在工程应用上参考现有自动化终止性工具与验证框架的实践经验，在真实项目中对“可用性（预算）/可靠性（保证）/覆盖（规模）”做系统评测 [7]。这些方向并不试图“推翻不可判定性”，而是探索在边界内如何把结论做得更强、更可用、更可审计。

10.4 课程思想在本文中的体现与收获

写完本文之后，我对课程里反复出现的一些“方法论”有了更具体的体会。其一，把现实问题改写成判定问题，并给出相应的形式语言表述，并不是为了追求抽象本身，而是为了把“讨论的对象”钉死：程序、输入、配置与转移各自对应什么，讨论才不会飘。其二，图灵机模型的价值在于统一：把现实程序映射到通用模型后，终止性检测的困难不是某门语言或某个工具链的局限，而是通用计算的边界。其三，归约把这些边界结论“搬运”到新问题上，让证明链条可以复用。

除此之外，贯穿全文的状态机视角也很有帮助：CFG、符号执行、抽象解释、运行时监控这些方法，表面上差异很大，但都能用“状态与转移”的语言来对照：它们各自压缩了什么、放弃了什么、又保证了什么。最终我的收获是：不可判定性并不是一句消极结论，它更像一条底线，逼着我们把工程目标说清楚，比如保证到什么程度、适用范围是什么、失败时怎么兜底，从而做出更稳健的系统设计。

参考文献

- [1] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012.
- [2] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.
- [3] Patrick Cousot. *Principles of Abstract Interpretation*. MIT Press, 2021.
- [4] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Yellow Paper*, 2014.
- [5] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2001.
- [6] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength SMT solver. *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 13243, pages 415–442, 2022.
- [7] Jera Hensel, Constantin Mensendiek, and Jürgen Giesl. AProVE: Non-termination witnesses for C programs (competition contribution). *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 13244, pages 403–407, 2022.
- [8] Ravindra Metta, Prasanth Yeduru, Hrishikesh Karmarkar, and Raveendra Kumar Medicherla. VeriFuzz 1.4: Checking for (Non-)termination (competition contribution). *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 13994, pages 594–599, 2023.
- [9] Simmo Saan, Julian Erhard, Michael Schwarz, Stanimir Bozhilov, Karoline Holter, Sarah Tilscher, Vesal Vojdani, and Helmut Seidl. Goblint: Abstract interpretation for memory safety and termination (competition contribution). *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 14572, pages 381–386, 2024.
- [10] Ravindra Metta, Hrishikesh Karmarkar, Kumar Madhukar, R. Venkatesh, and Supratik Chakraborty. PROTON: PRObes for termination or not (competition

- contribution). *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 14572, pages 393–398, 2024.
- [11] Arash Sabbaghi and Mohammad Reza Keyvanpour. A systematic review of search strategies in dynamic symbolic execution. *Computer Standards & Interfaces*, 72:103444, 2020.
- [12] Akihisa Yamada. Termination of term rewriting: Foundation, formalization, implementation, and competition (invited talk). In *International Conference on Formal Structures for Computation and Deduction (FSCD)*, LIPIcs 260, pages 4:1–4:5, 2023.

致谢

首先衷心感谢孙岩炜老师在本学期《计算理论》课程中的系统讲授。老师在课堂上围绕“计算模型、形式语言、可计算性/复杂性”的主线，从有穷自动机与正则语言、上下文无关文法与下推自动机，到图灵机、可判定性与不可判定性，再到复杂性理论的归约问题，帮助我建立了理解计算边界的一套统一视角。

本文选取“程序自动终止性检测”这一现实系统问题，正是希望把课堂中反复强调的方法落实到一个具体场景中：先用形式化语言把问题表达为判定问题，再用图灵机模型与归约证明给出能力边界，最后回到工程实践讨论在边界内如何做取舍，例如受限模型、保守近似与资源有界化。在写作过程中，我对“不可判定性并非否定一切，而是指导我们如何设定可实现目标”有了更清晰的体会。

感谢老师的课件资料与计算理论教材，以及计算理论相关论文资料为本文提供的论证依据与背景知识。以上帮助共同促成了本文的完成。

A 符号与缩写

为便于阅读与交叉引用，本节汇总正文中出现的主要缩写与记号。

缩写

HALT_{TM}	图灵机停机问题 (Halting Problem)
PTP	程序实例终止性判定问题 (Program Termination Problem)
TOT	总终止性问题 (对所有输入终止)
BPTP	有界终止性问题 (k 步内终止)
P / NP	复杂性类: 多项式时间 / 非确定性多项式时间
IDE	集成开发环境 (Integrated Development Environment)
CI/CD	持续集成/持续交付 (Continuous Integration/Delivery)
WCET	最坏情况执行时间 (Worst-Case Execution Time)
CFG	控制流图 (Control Flow Graph)
SMT	可满足性模理论 (Satisfiability Modulo Theories)
Soundness	可靠性/正确性 (不把不终止误判为终止)
Completeness	完备性 (终止时一定能判为终止)

记号

P, x	程序与其输入 (正文默认 P 为确定性程序实例)
$\langle P, x \rangle$	将程序与输入编码为有限串的有效编码
M_P / w_x	由 P 构造的图灵机 / 输入 x 的编码串
$P(x) \downarrow / P(x) \uparrow$	终止 / 不终止 (或未在有限步内停机)
$C_0(P, x), C_h$	初始配置 / 终止 (停机) 配置
$\vdash / \vdash^*$	一步转移 / 有限步可达 (转移闭包)
(pc, σ)	程序配置: 程序位置 pc 与存储环境 σ
$\sigma[x \mapsto v]$	将变量 x 更新为 v 的环境
φ	符号执行中的路径条件 (约束公式)
$A \leq_m B$	多对一归约 (many-one reduction)
$S, \hat{S}$	具体状态空间与抽象状态空间
α, γ	抽象/具体化函数 (Galois 连接语境下)
$\hat{X} = \hat{F}(\hat{X})$	抽象解释中的不动点方程

B Toy 语言补充：语法与语义要点

为便于复现实验，本附录补充本文 toy 语言 $\mathcal{L}_{toy}$ 的核心语法与运行配置表示。

- **配置：** (pc, σ) ，其中 pc 为程序计数器， σ 为变量到整数的映射；
- **一步执行：** $(pc, \sigma) \rightarrow (pc', \sigma')$ 由当前指令语义确定；
- **终止：** 当 pc 指向程序末尾（或显式 `return`）时视为到达终止配置。

在该模型下，“终止性”与“可达终止配置”是一致的，从而便于与前文状态机视角下的可达性表述对应。

B.1 语法（BNF）与示例程序

本文在正文中使用的 $\mathcal{L}_{toy}$ 可视为一个极简命令式语言。为了便于实现解释器，可以把语法写成更细化的 BNF 形式：

$$\begin{aligned} S &::= \text{skip} \mid S_1; S_2 \mid x := e \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \mid \text{while } b \text{ do } S, \\ e &::= c \mid x \mid e_1 + e_2 \mid e_1 - e_2, \\ b &::= e_1 > 0 \mid e_1 = 0. \end{aligned}$$

其中 x 为变量名， c 为整数常量。下面给出一个“单调下降”的经典终止模式示例：

```

1 // while(x > 0) x := x - 1
2 while (x > 0) do
3     x := x - 1;
```

Listing 5: Toy 示例：单调下降循环（容易证明终止）

B.2 小步操作语义

解释器实现通常采用小步语义：每一步只执行一个原子动作，使得“执行 = 状态转移”在形式上更清晰。以赋值与顺序组合为例，可以写成如下推导规则（示意）：

$$\frac{}{(x := e, \sigma) \rightarrow (\text{skip}, \sigma[x \mapsto \llbracket e \rrbracket_\sigma])} \quad \frac{(S_1, \sigma) \rightarrow (S'_1, \sigma')}{(S_1; S_2, \sigma) \rightarrow (S'_1; S_2, \sigma')}$$

其中 $\llbracket e \rrbracket_\sigma$ 表示表达式在环境 σ 下的求值结果。循环语句可以被“展开”为条件分支，从而把 `while` 的语义化为若干步的 `if` 与顺序组合。

为了让 toy 语义更完整，这里再补充条件语句与循环的典型规则（示意）：

$$\frac{\llbracket b \rrbracket_{\sigma} = \text{true}}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \rightarrow (S_1, \sigma)} \quad \frac{\llbracket b \rrbracket_{\sigma} = \text{false}}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \rightarrow (S_2, \sigma)}$$

$$(\text{while } b \text{ do } S, \sigma) \rightarrow (\text{if } b \text{ then } (S; \text{while } b \text{ do } S) \text{ else skip}, \sigma).$$

这些规则体现了一个关键点：**while** 循环并不是“神秘的无限结构”，而是可以被系统化地改写成条件分支与顺序组合，从而让循环的行为完全落入“一步一步的状态转移”框架之中。也正因如此，终止性问题才能自然地表达为“是否存在无限长的推导序列”。

要点：为什么小步语义适合讨论终止性

小步语义把执行过程离散化为一步步的转移，使“终止”对应于到达无后继的状态，而“不终止”对应于存在无限转移序列。这样，终止性分析就能与状态机、可达性、循环检测等概念自然对接，也更便于把工程工具（CFG/符号执行/抽象解释/监控）统一到同一语言里比较。